

GFS, MapReduce, and Spark

DS 5110: Big Data Systems
Summer 2026
Lecture 4

Yue Cheng



UNIVERSITY
of VIRGINIA

Some material taken/derived from:

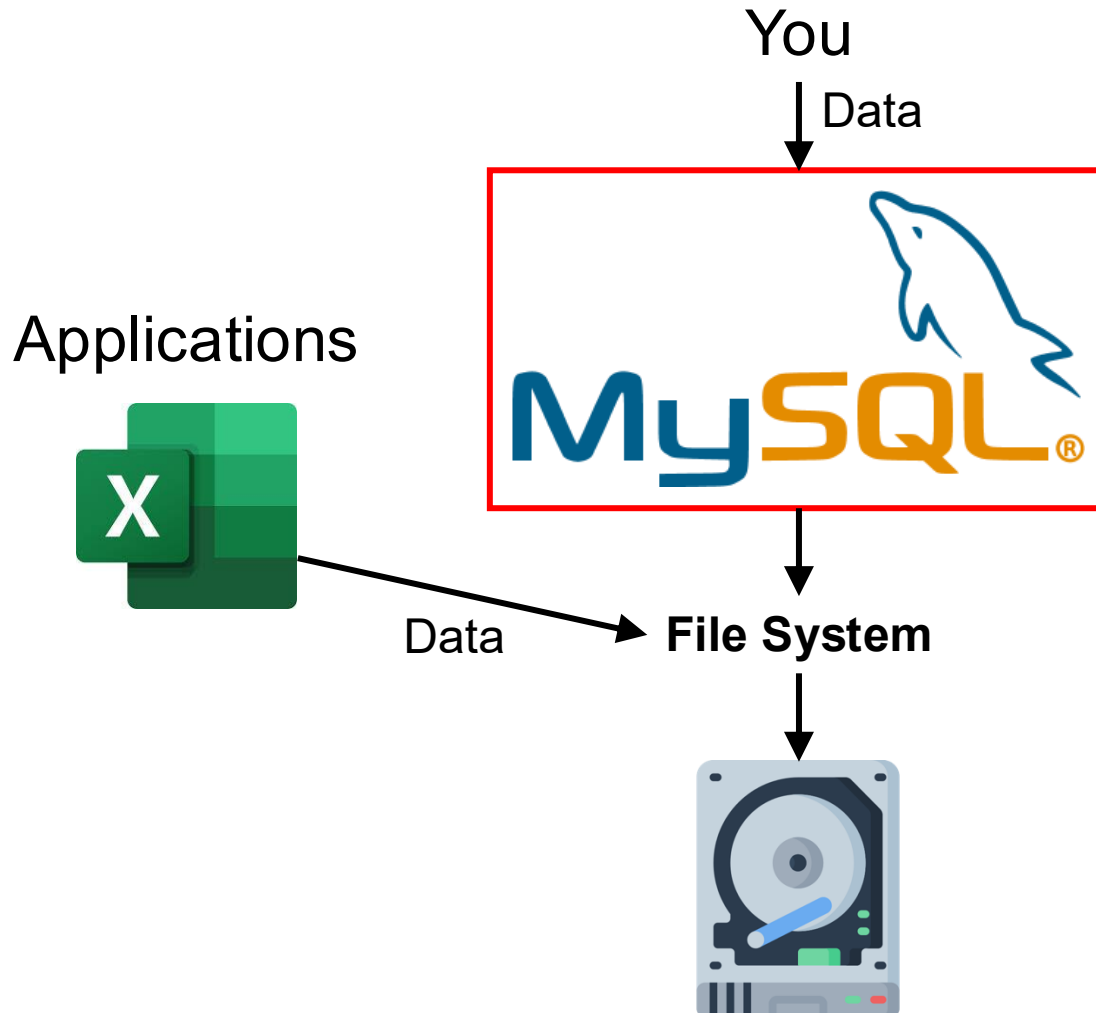
- Wisconsin CS 320 by Tyler Caraza-Harter.

@ 2026 released for use under a [CC BY-SA](https://creativecommons.org/licenses/by-sa/4.0/) license.

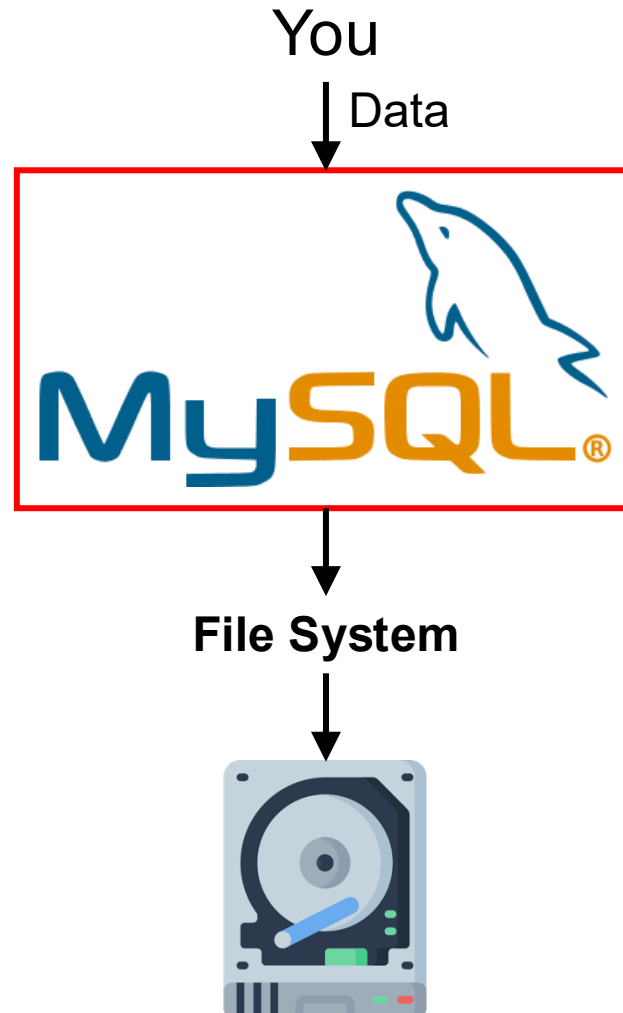
Outline

- GFS (HDFS)
- MapReduce
- Spark RDD

Design: Storage systems are generally built as a composition of layered subsystems

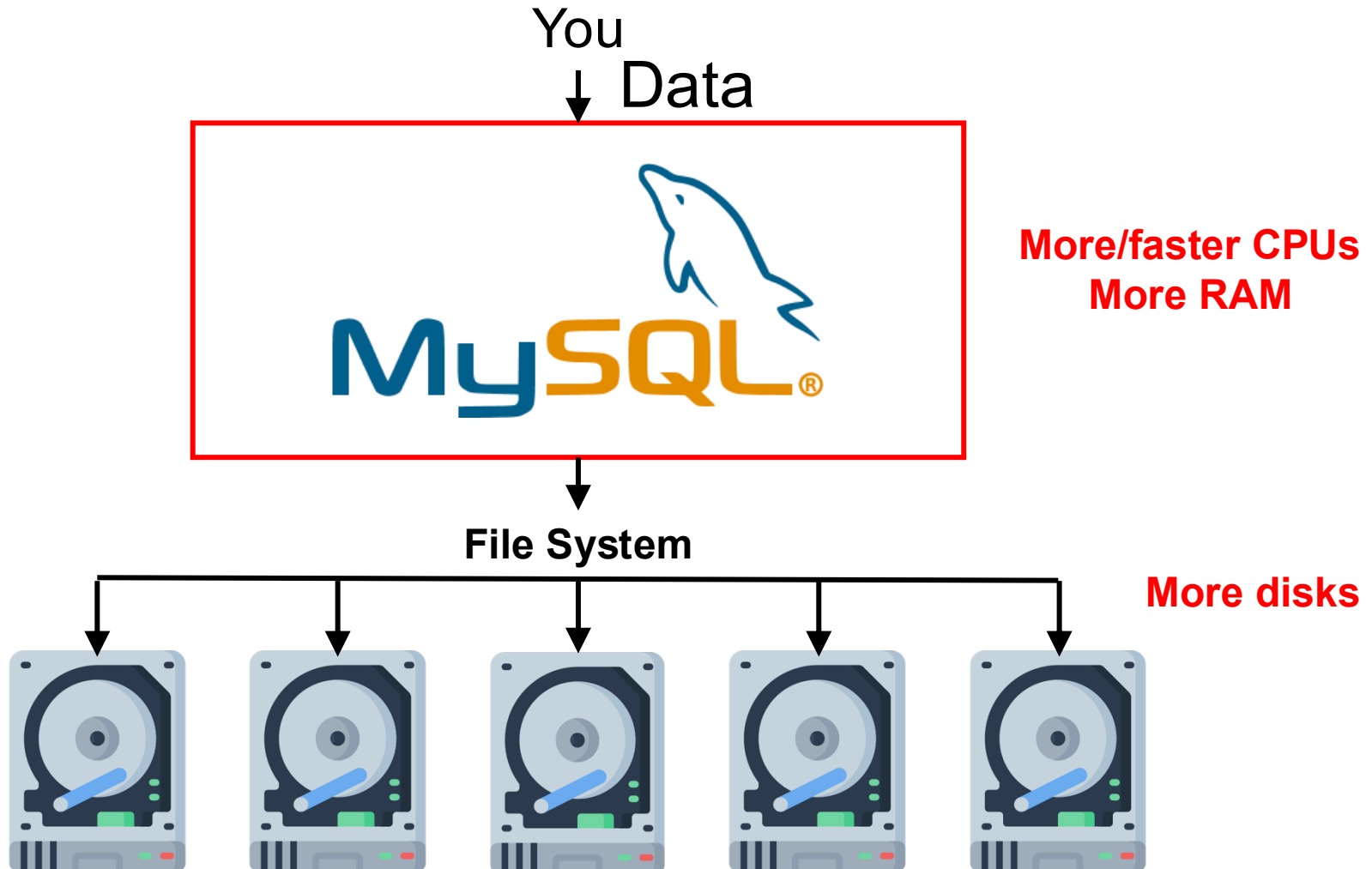


Problem: What if your data is too big?



Problem 1: What if your data is too big?

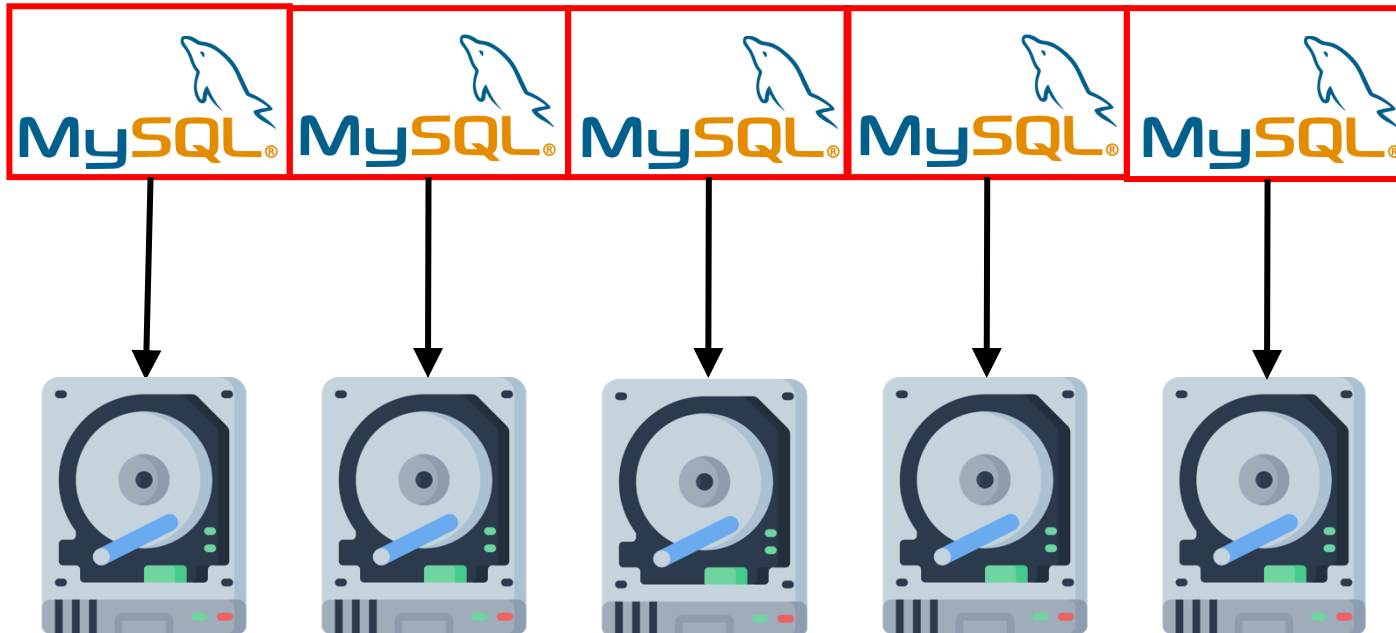
Option 1: **Scale up** (buy better hardware)



Problem 1: What if your data is too big?

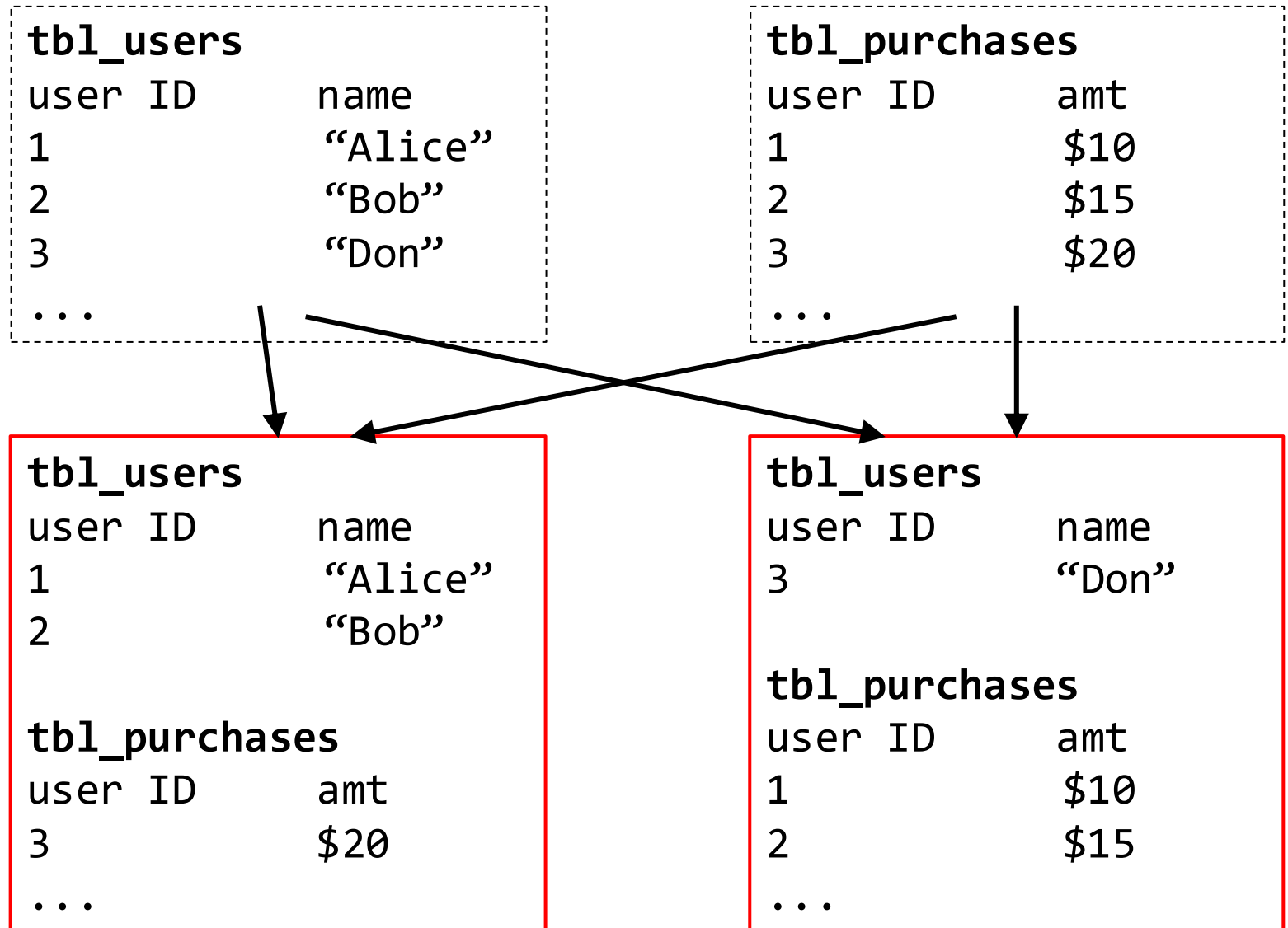
Option 2: **Scale out** (more machines)

Where does the data actually go?



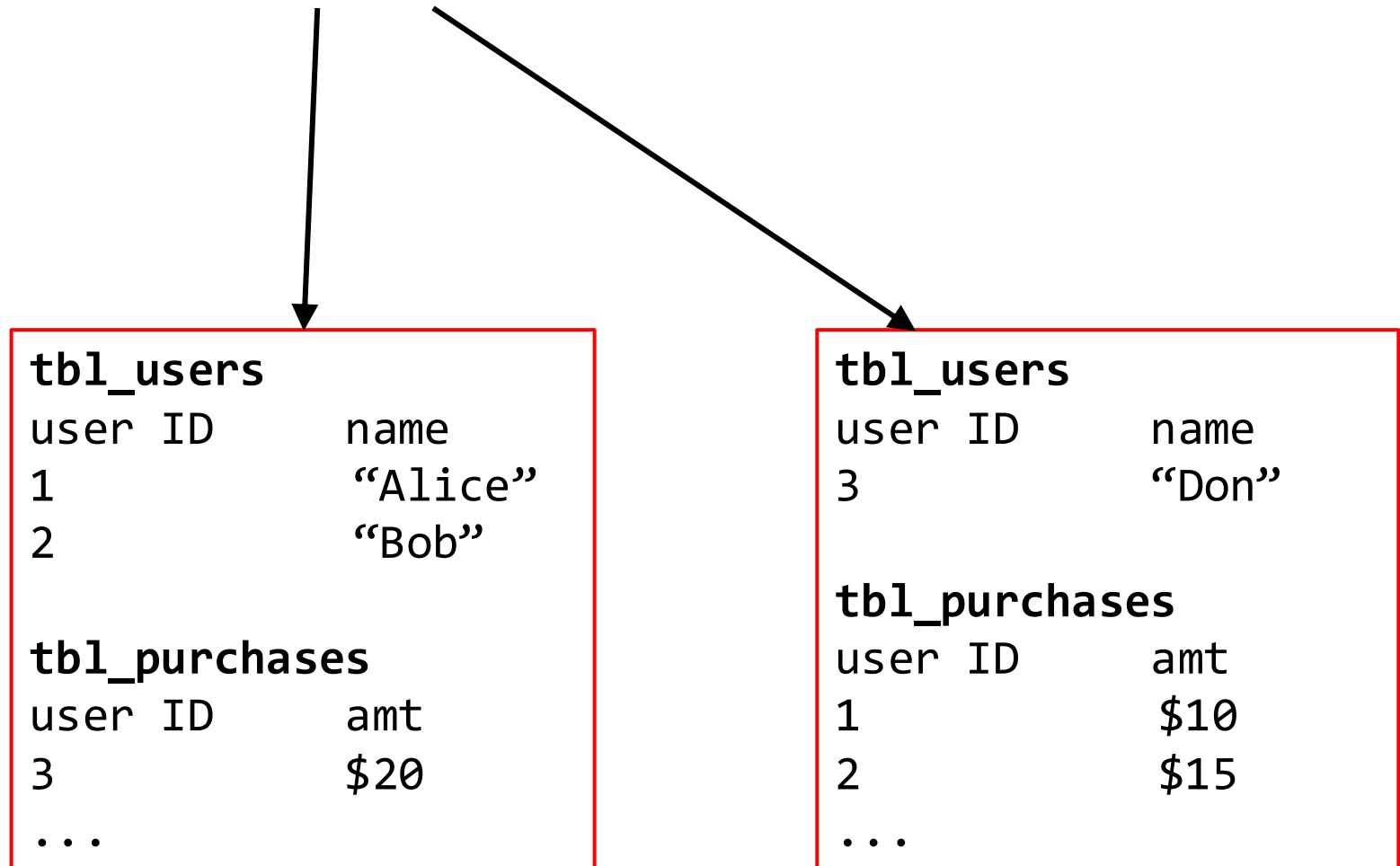
...1000 more...

Approach: Partition the data



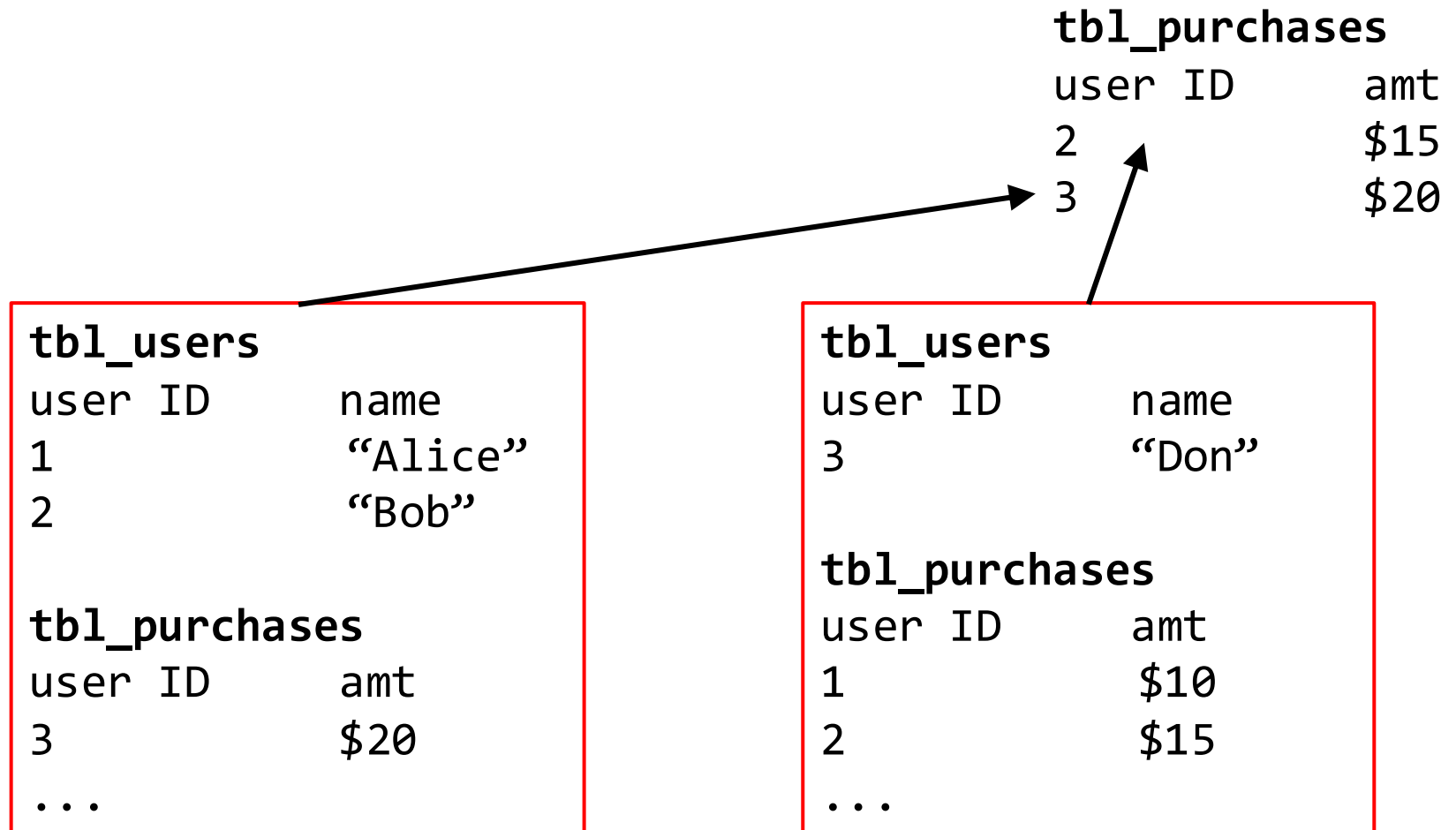
Approach: Send queries to multiple DBs

```
SELECT * FROM tbl_purchases WHERE amt > 12
```



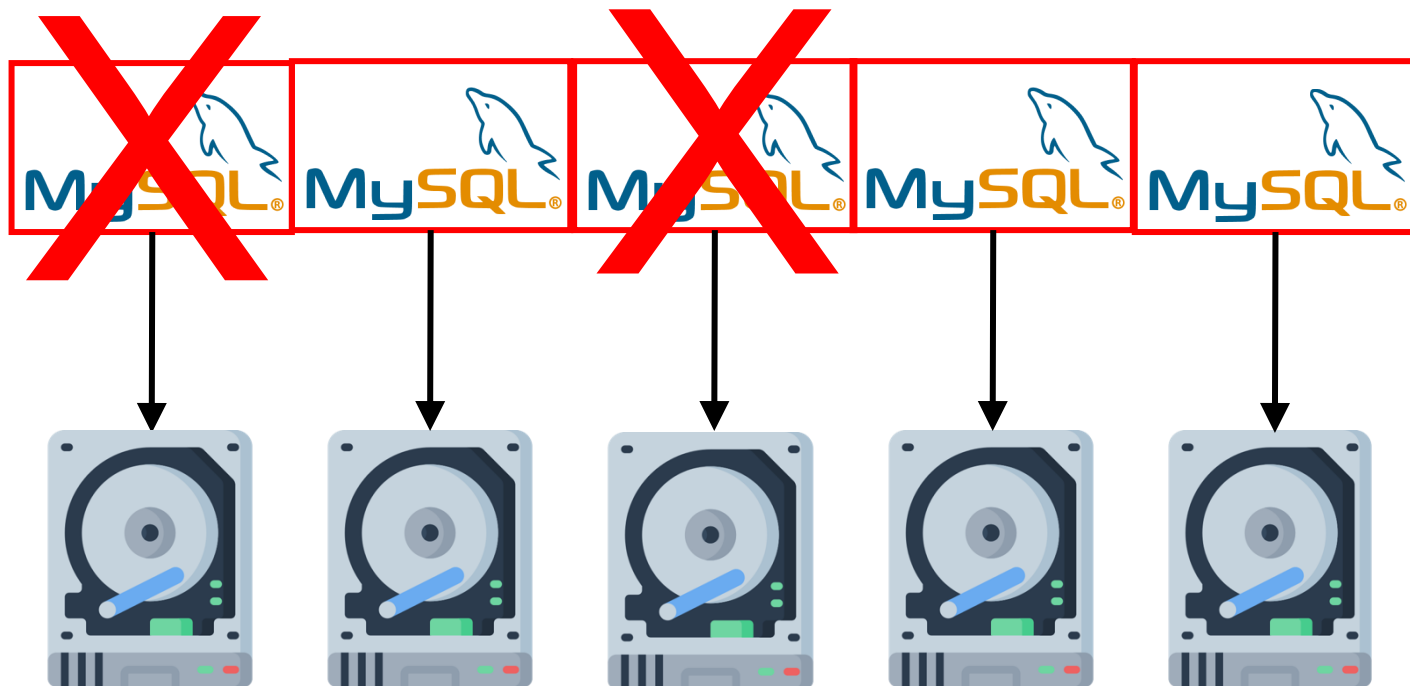
... Combine results

```
SELECT * FROM tbl_purchases WHERE amt > 12
```



Problem 2: What if your server dies?

Happens all the time when you have 1000s of machines...

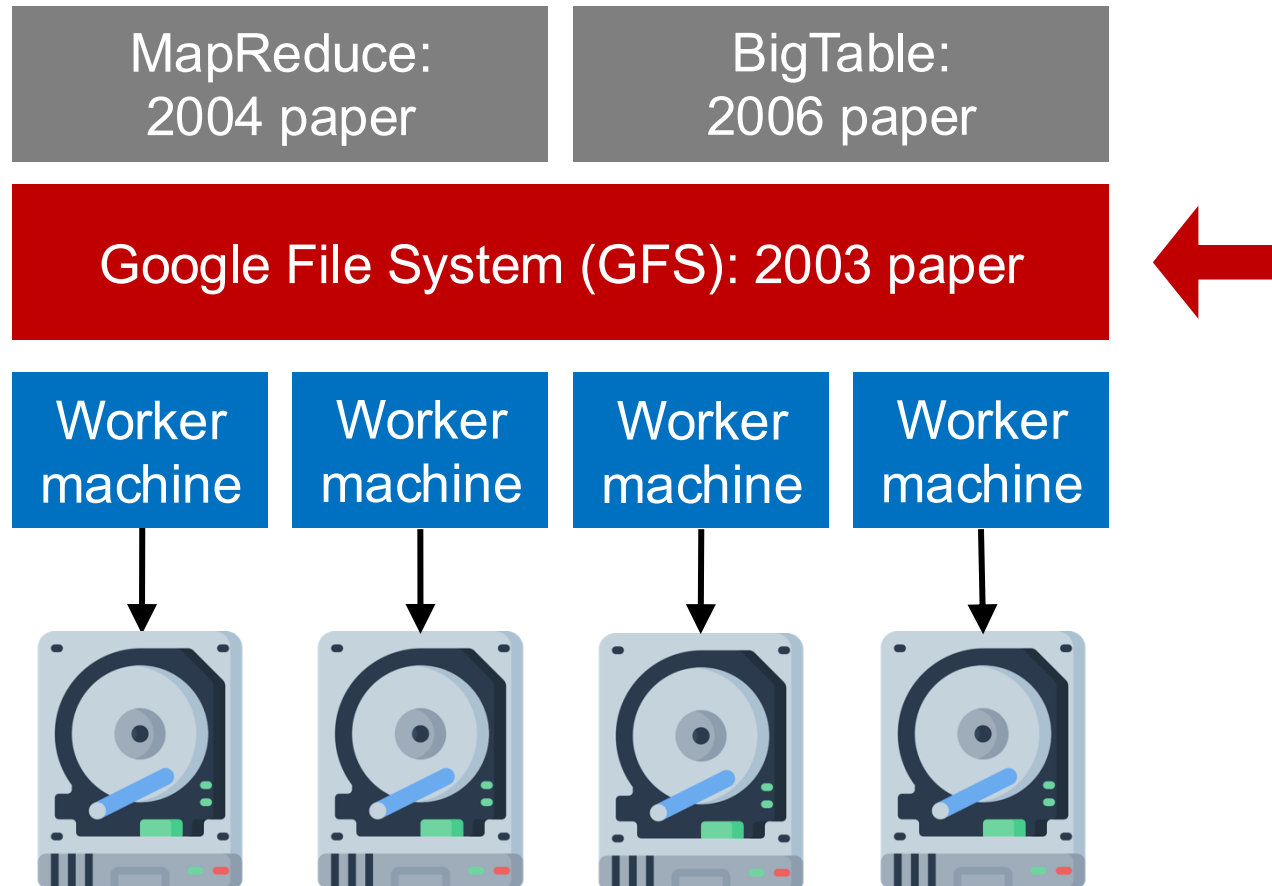


...1000 more...

Motivation for large DFS (GFS / HDFS)

- Scaling to many machines is essential
- Fault tolerance is essential

Google big data infrastructure



Radical idea: base everything on lots of cheap, commodity hardware

Hadoop ecosystem

Yahoo, Facebook, Cloudera, and others developed open-source Hadoop ecosystem, mirroring Google's big data systems

	Google (paper only)	Hadoop (open source)	Modern Hadoop
Distributed File System	GFS	HDFS	
Distributed Processing & Analytics	MapReduce	Hadoop MapReduce	Spark
Distributed Database	BigTable	HBase	MongoDB

<https://hadoop.apache.org/>

HDFS: DataNodes store file blocks

F1: "ABCD"

F2: "EFGHIJKL"

Some file fits in a single block

"ABCD" (F1.1)

**DataNode
Computers**



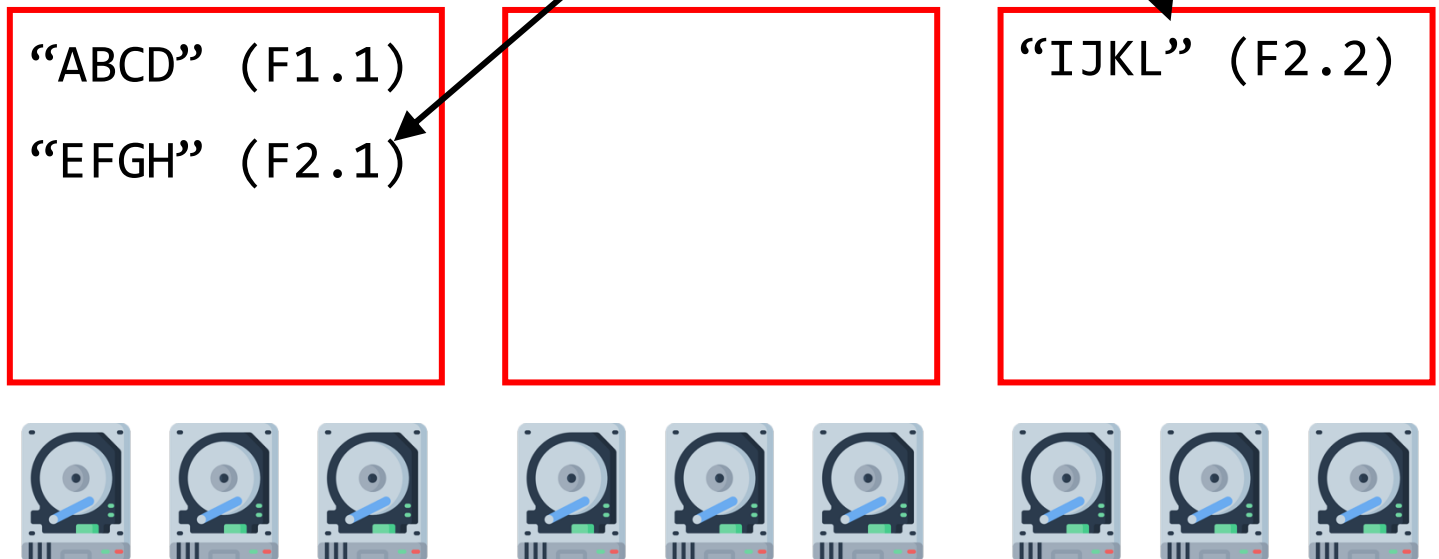
HDFS: Partitioning across DataNodes

F1: "ABCD"

F2: "EFGHIJKL"

Bigger files are **partitioned**
across multiple DataNodes

**DataNode
Computers**



HDFS: Replication across DataNodes

F1: “ABCD”

3x replication

F2: “EFGHIJKL”

2x replication

**DataNode
Computers**

“ABCD” (F1.1)

“EFGH” (F2.1)

“ABCD” (F1.1)

“IJKL” (F2.2)

“IJKL” (F2.2)

“ABCD” (F1.1)

“EFGH” (F2.1)



HDFS: Replication across DataNodes

F1: “ABCD”

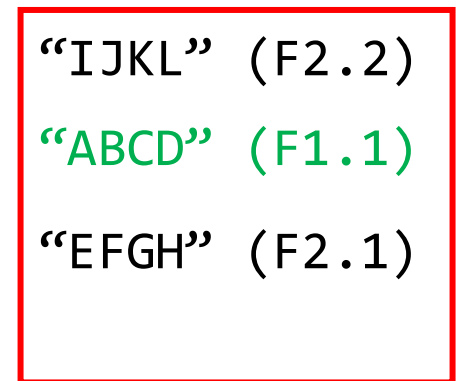
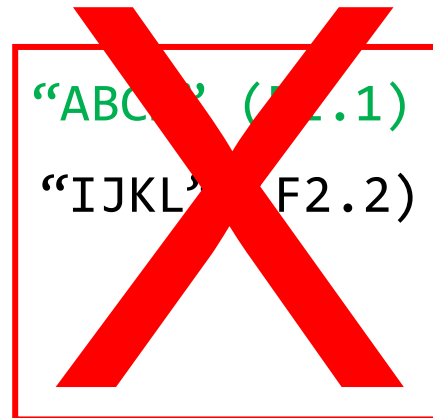
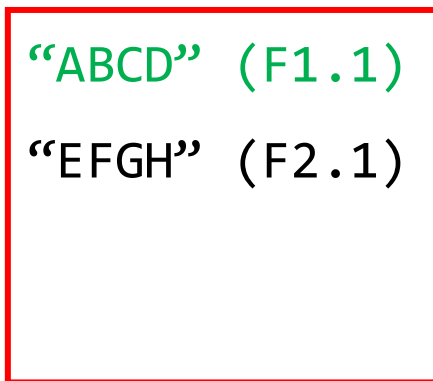
3x replication

F2: “EFGHIJKL”

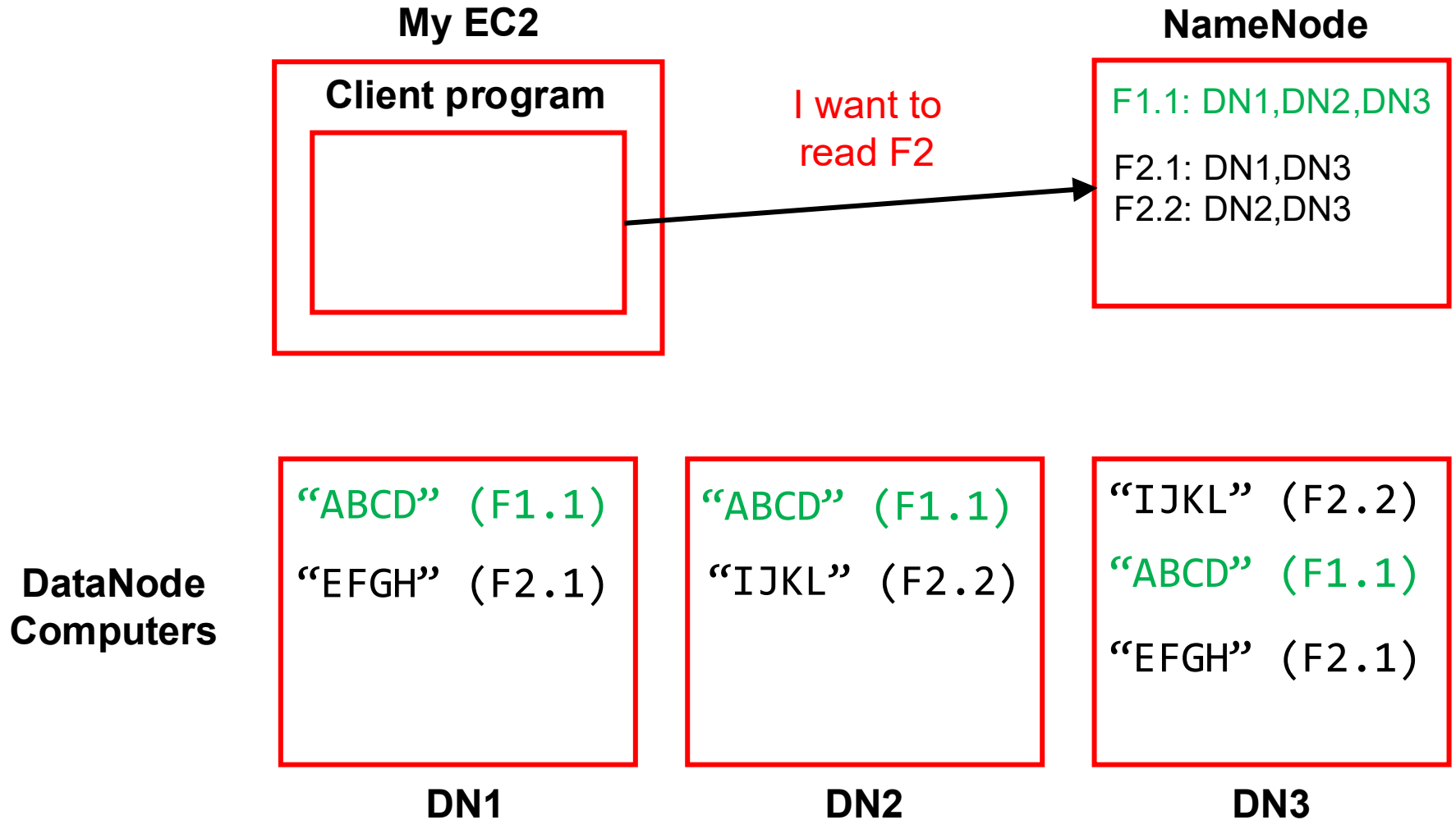
2x replication

If a DataNode dies, we still have all the data.
Which file is safer in general? F1 or F2?

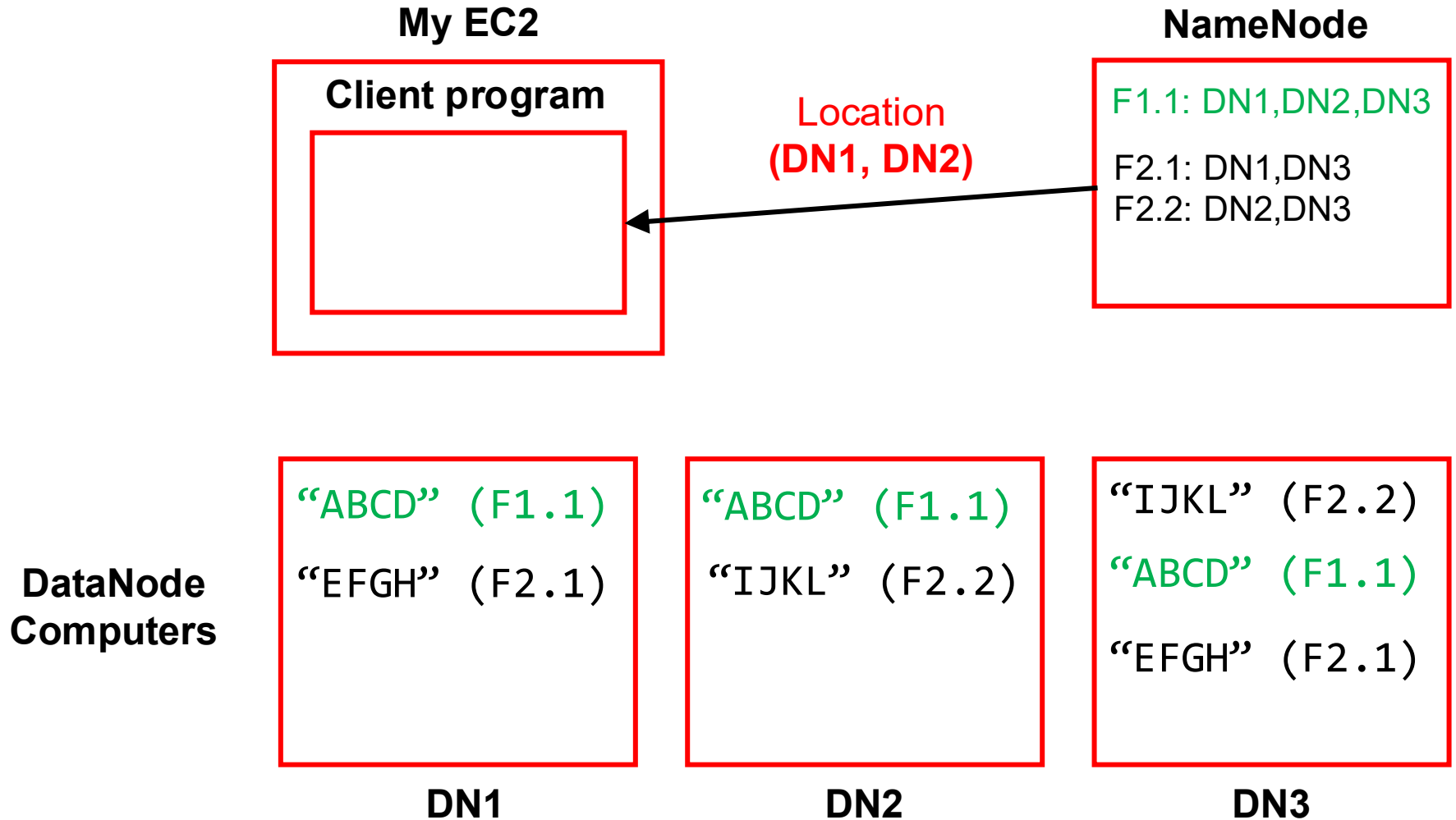
**DataNode
Computers**



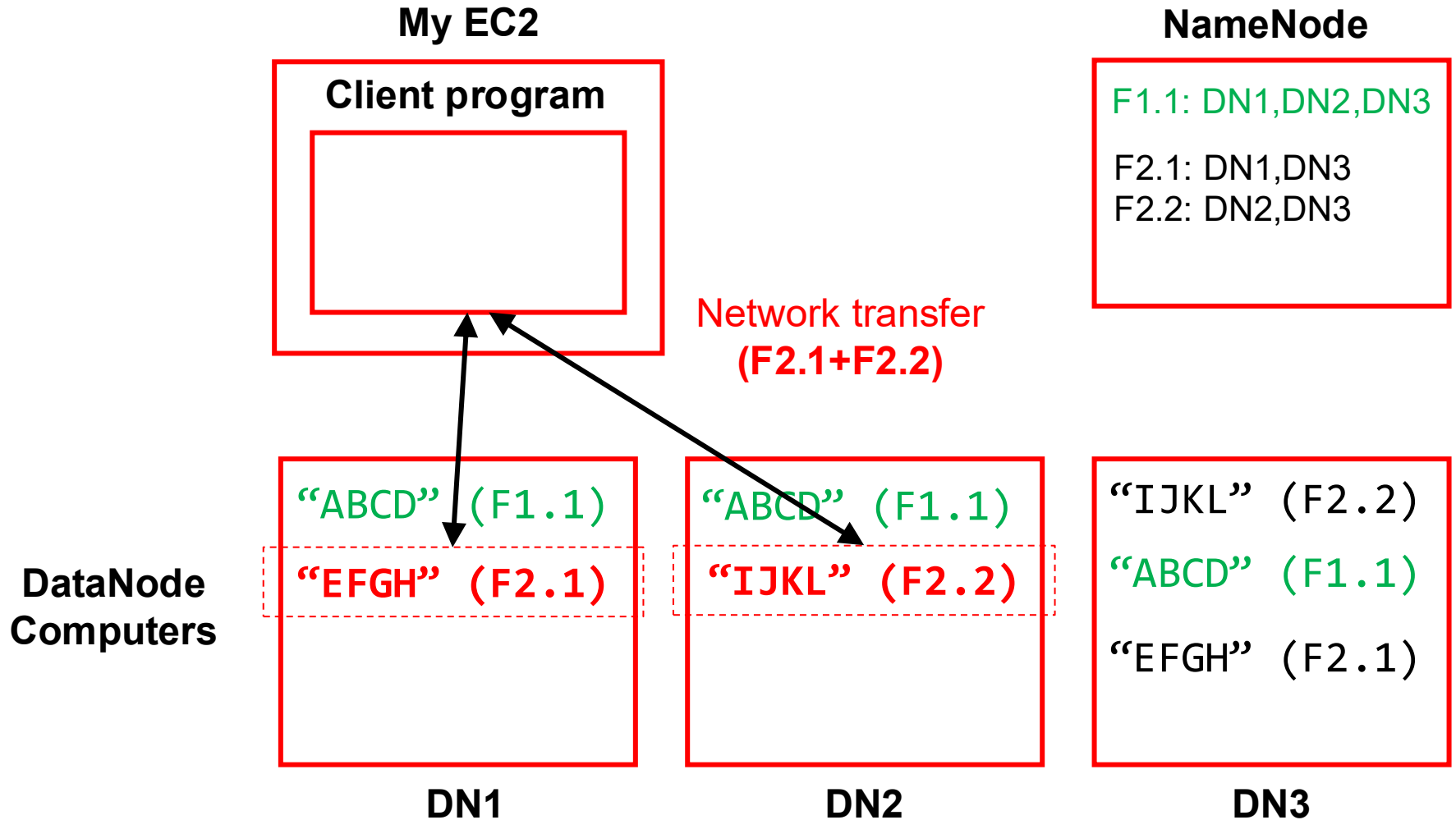
NameNode/Worker architecture



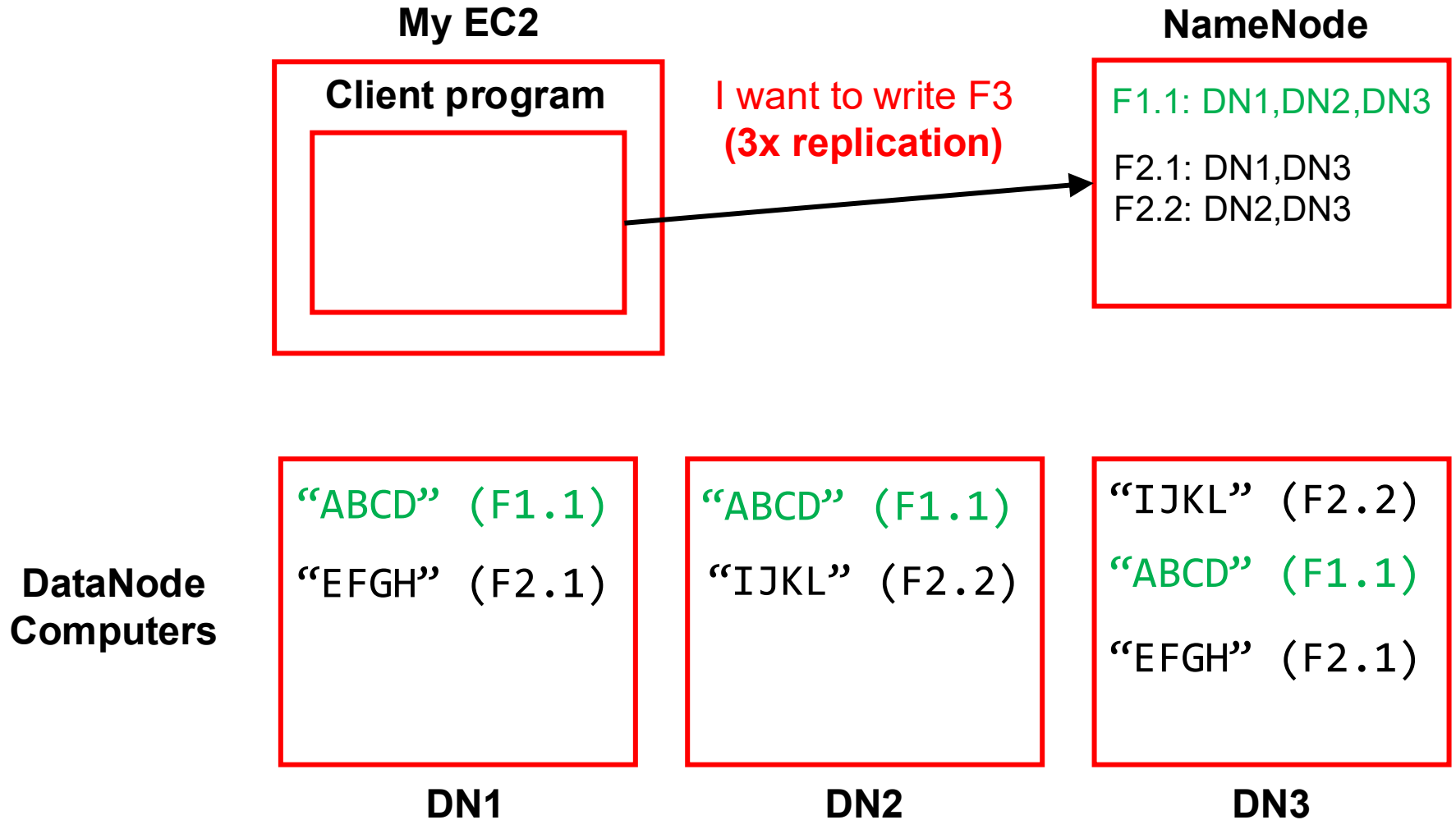
NameNode/Worker architecture



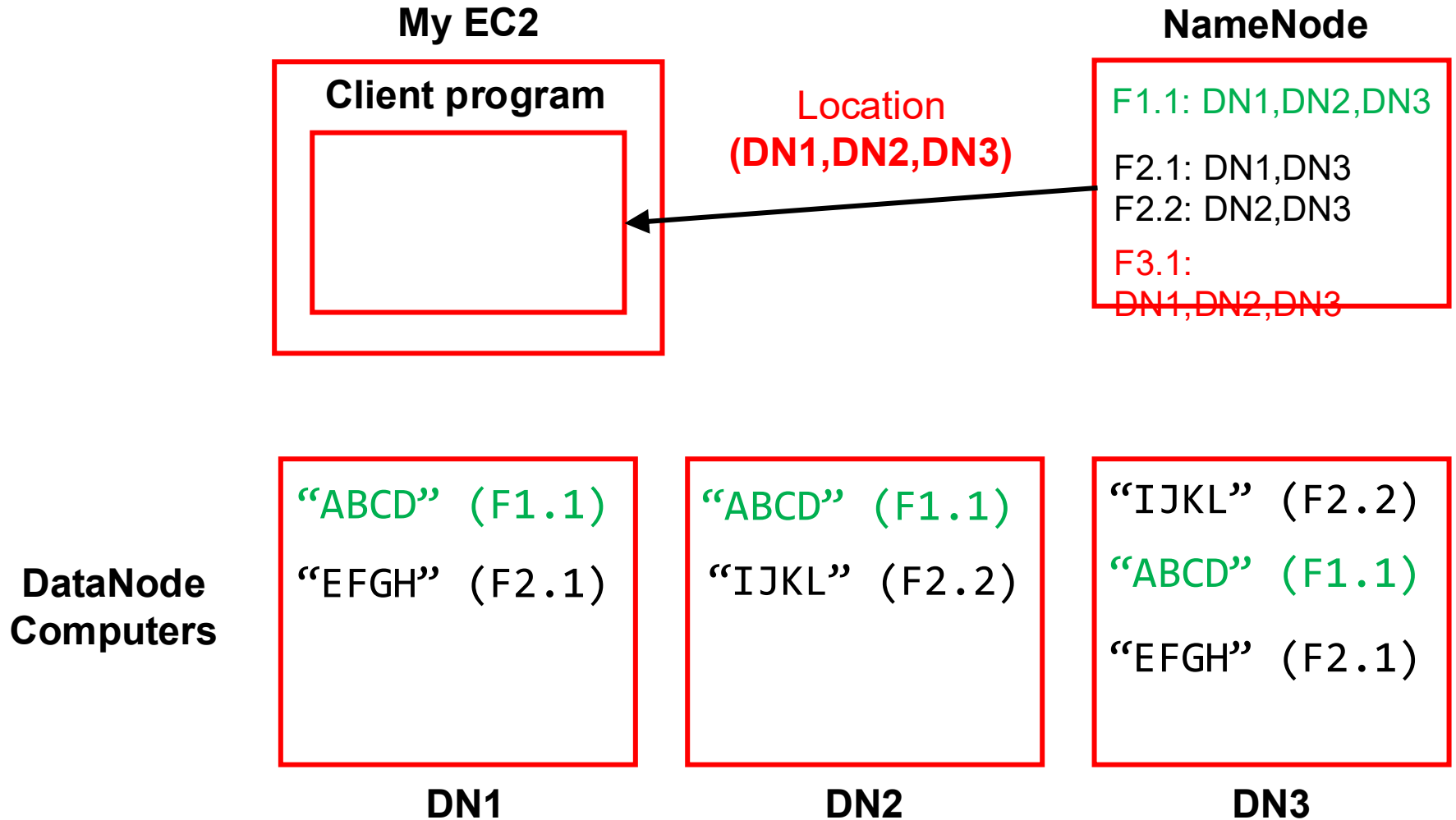
NameNode/Worker architecture



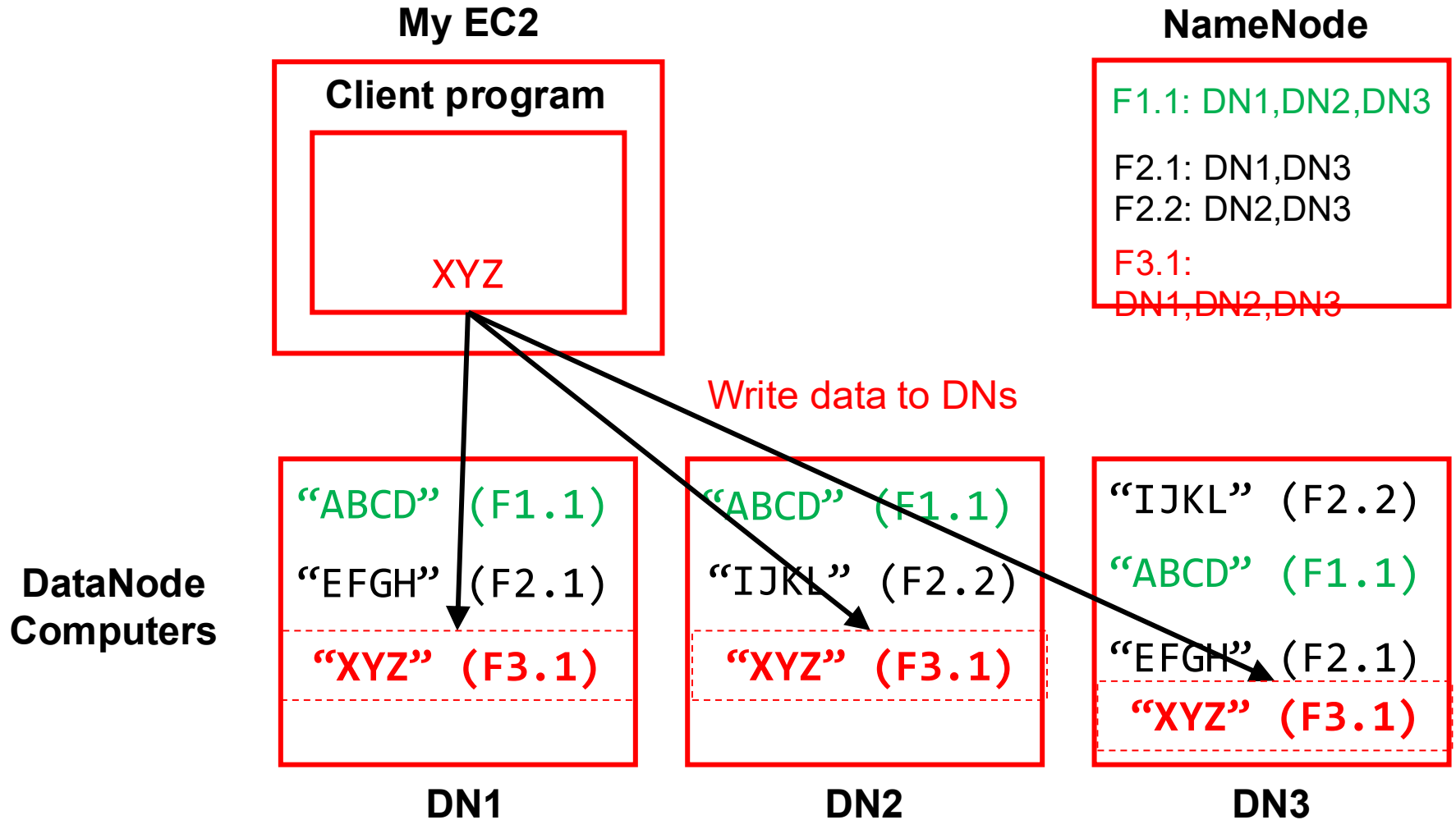
NameNode/Worker architecture



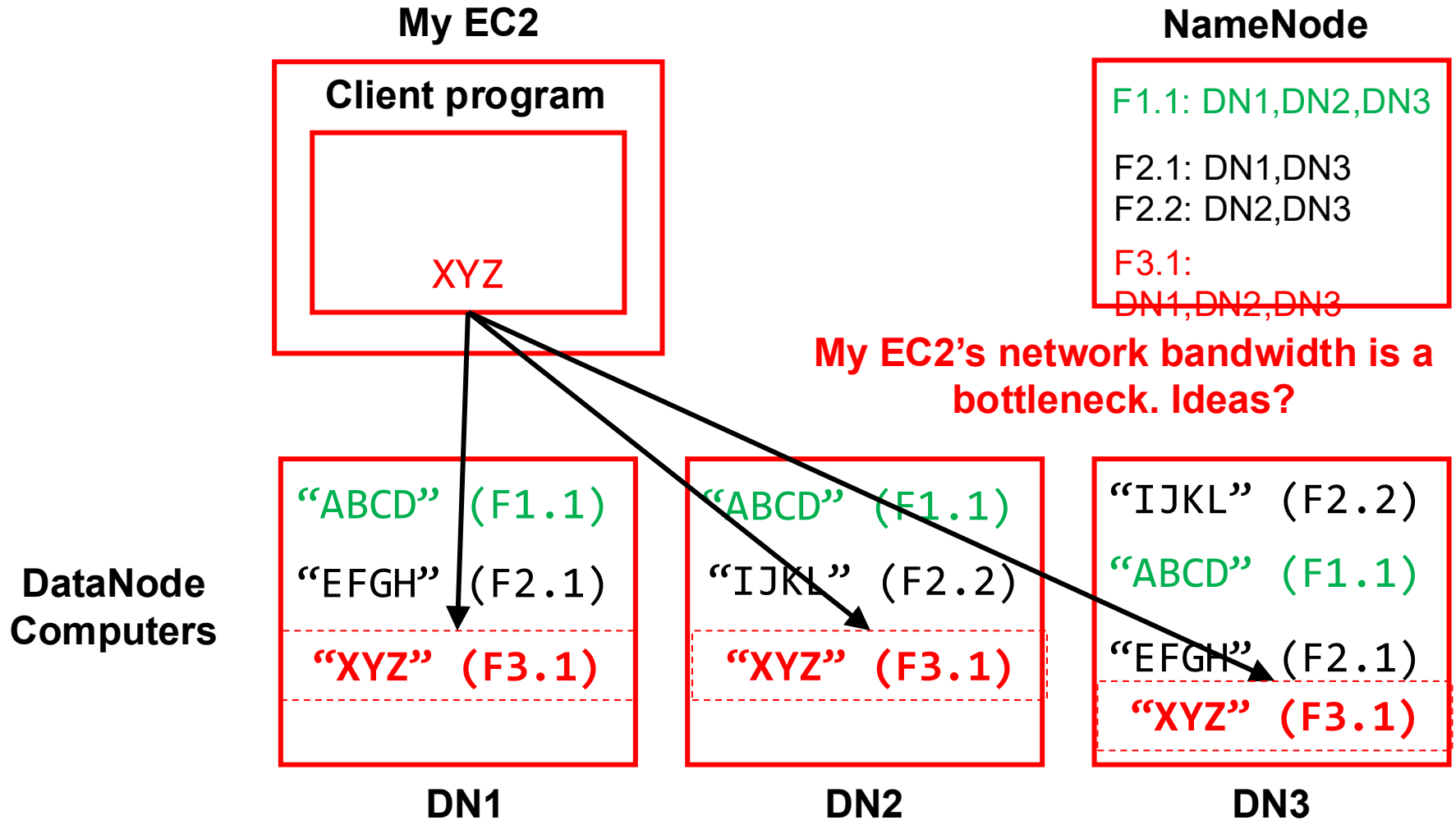
NameNode/Worker architecture



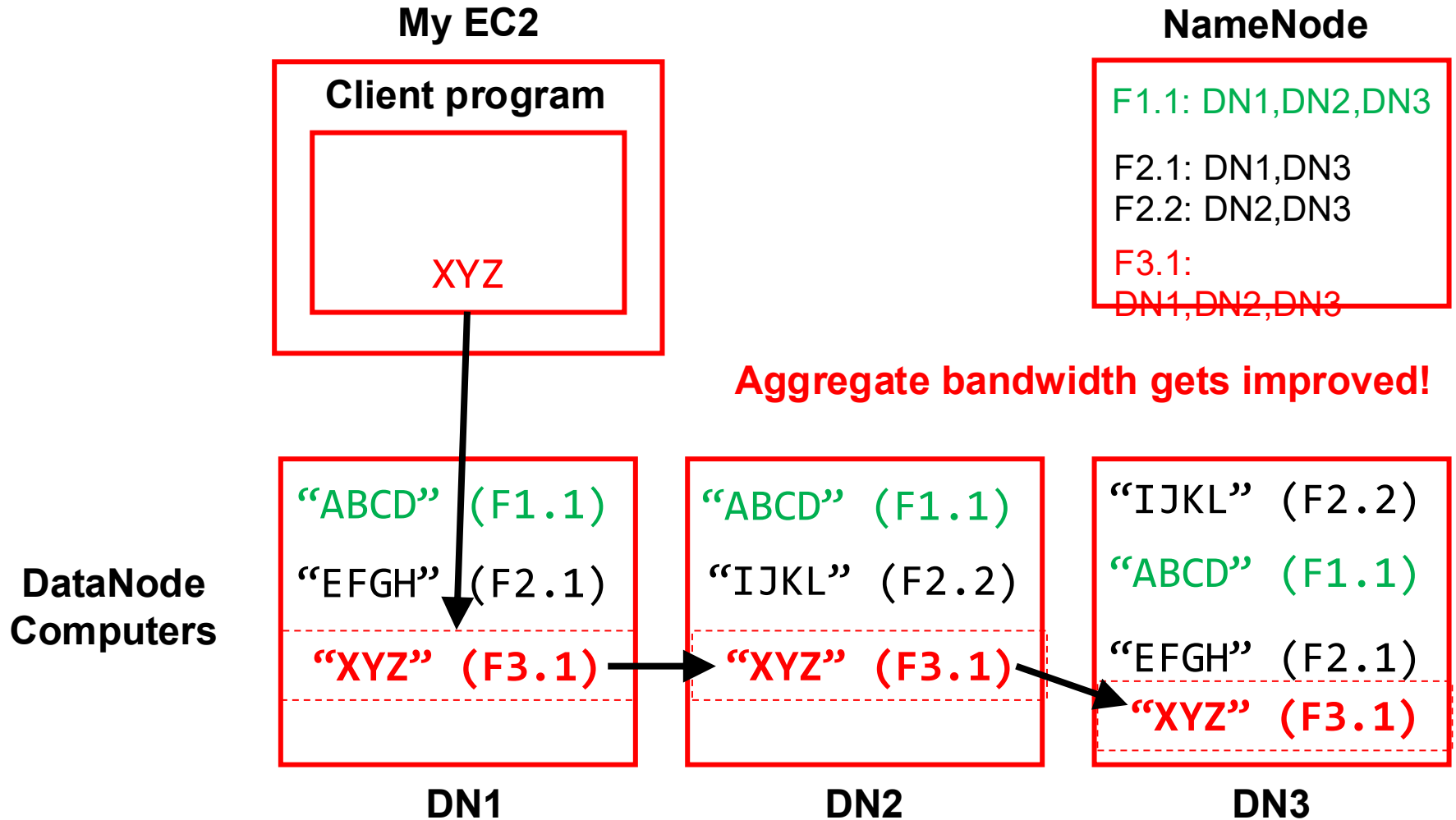
NameNode/Worker architecture



NameNode/Worker architecture



Pipelined writes



How are reads/writes amplified at disk level?

Q1: If a client **writes** 4MB to a 2x replicated file, how much data does HDFS **write** to disks?

Q2: If a client **reads** 2MB from a 3x replicated file, how much data do we **read** from disks?

NameNode

F1.1: DN1,DN2,DN3

F2.1: DN1,DN3

F2.2: DN2,DN3

F3.1:

~~DN1,DN2,DN3~~

DataNode Computers

“ABCD” (F1.1)

“EFGH” (F2.1)

“XYZ” (F3.1)

DN1

“ABCD” (F1.1)

“IJKL” (F2.2)

“XYZ” (F3.1)

DN2

“IJKL” (F2.2)

“ABCD” (F1.1)

“EFGH” (F2.1)

“XYZ” (F3.1)

DN3

What are the tradeoffs of replication factor and block size?

Benefit of high replication?

Benefit of low replication?

Benefit of large block size?

Benefit of small block size?

NameNode

F1.1: DN1,DN2,DN3

F2.1: DN1,DN3

F2.2: DN2,DN3

F3.1:

~~DN1,DN2,DN3~~

DataNode
Computers

“ABCD” (F1.1)

“EFGH” (F2.1)

“XYZ” (F3.1)

DN1

“ABCD” (F1.1)

“IJKL” (F2.2)

“XYZ” (F3.1)

DN2

“IJKL” (F2.2)

“ABCD” (F1.1)

“EFGH” (F2.1)

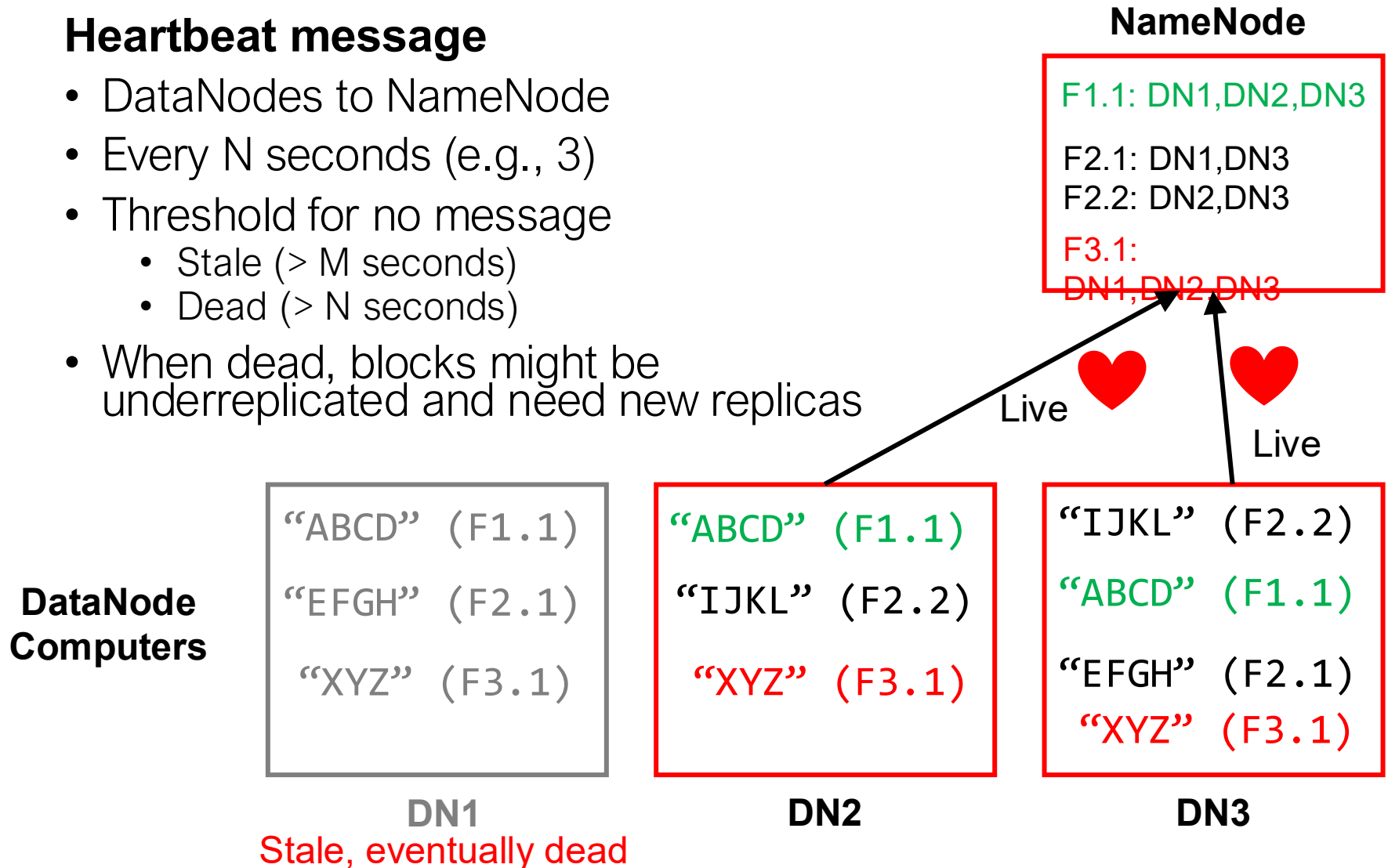
“XYZ” (F3.1)

DN3

How do we know when a DataNode fails?

Heartbeat message

- DataNodes to NameNode
- Every N seconds (e.g., 3)
- Threshold for no message
 - Stale (> M seconds)
 - Dead (> N seconds)
- When dead, blocks might be underreplicated and need new replicas



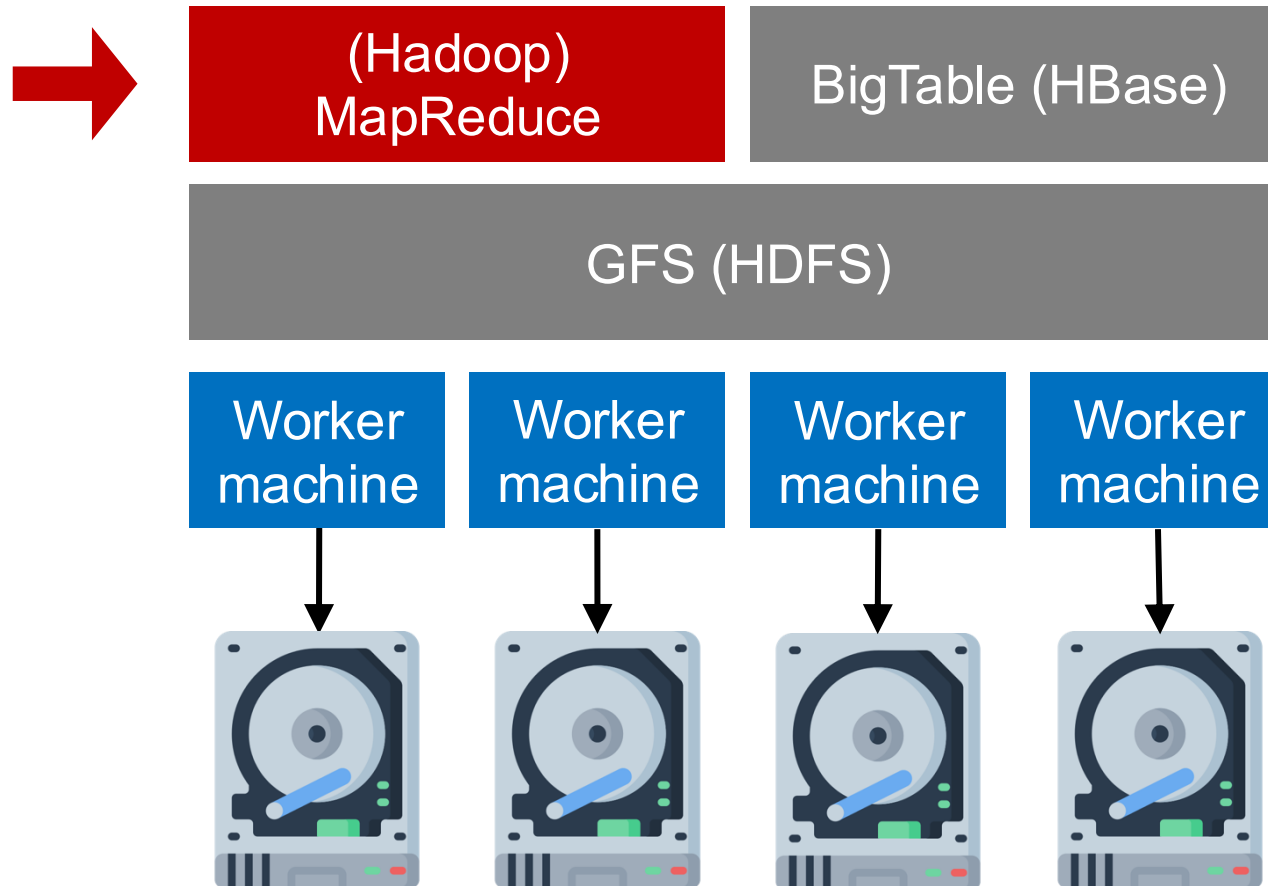
Summary: Some key ideas applied to GFS/HDFS

- To build complex systems...
- To scale out...
- To handle faults...
- To detect faults...
- To optimize I/O...

Outline

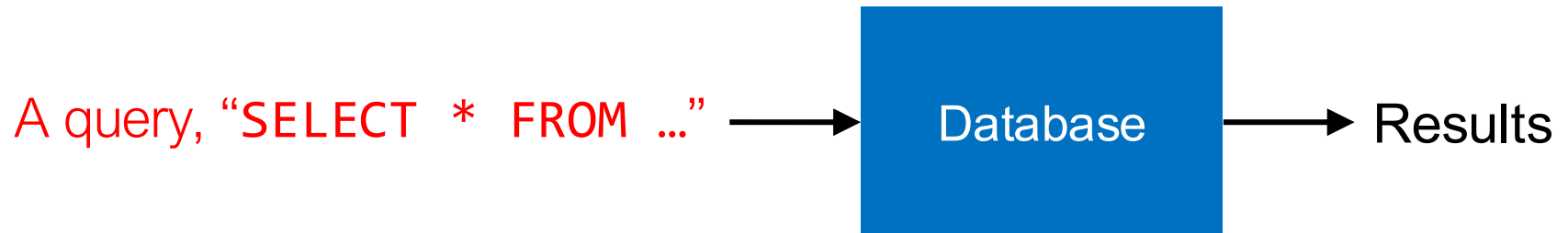
- GFS (HDFS)
- MapReduce
- Spark RDD

MapReduce

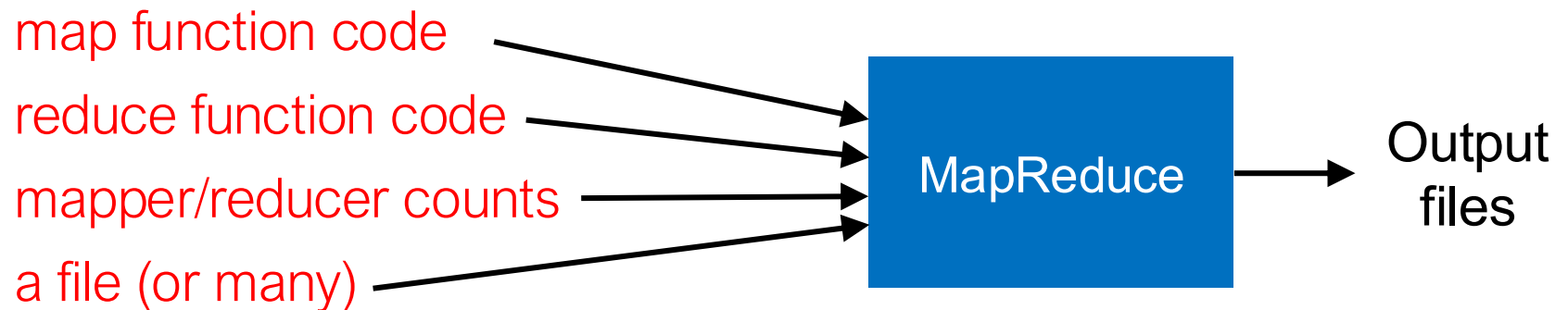


How does big data system answer questions?

SQL

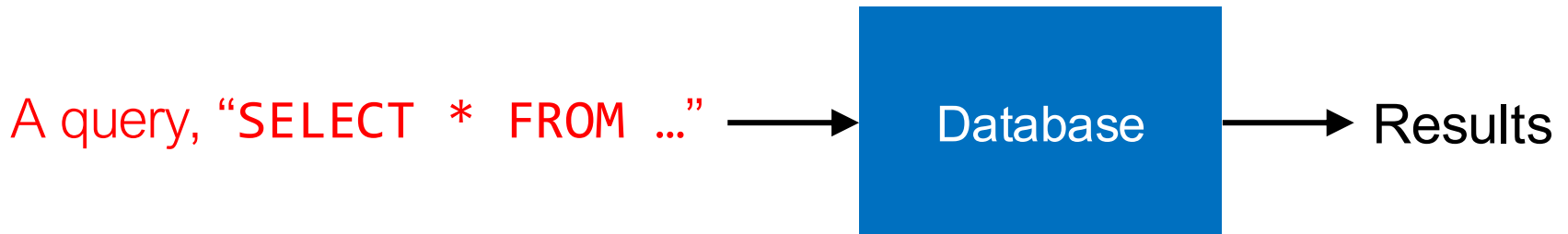


MapReduce

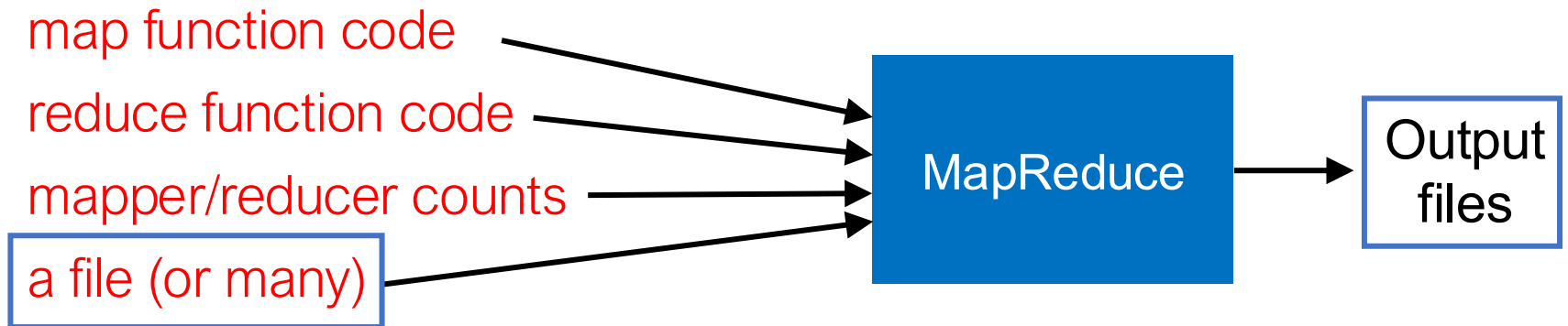


How does big data system answer questions?

SQL



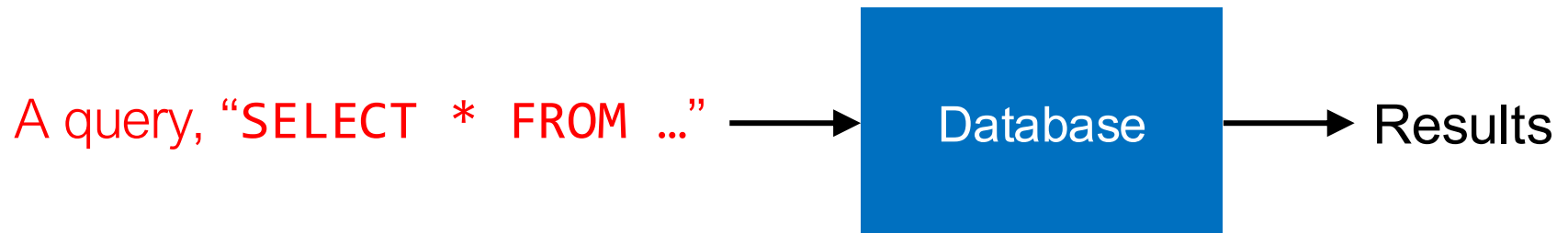
MapReduce



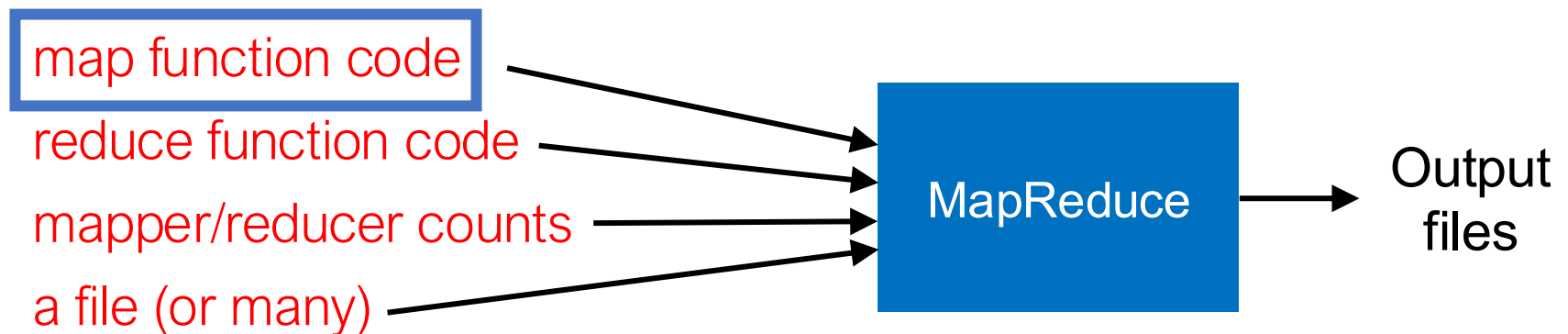
Input/output files are generally stored in HDFS

How does big data system answer questions?

SQL



MapReduce



Mappers by example: What are the colors of the squares?

input.csv (in HDFS)

color,	shape,	size
red,	circle,	3
red,	square,	5
blue,	oval,	1
green,	square,	3

```
def map(key, value):  
    ...
```

In SQL:

SELECT color FROM table WHERE shape = “square”

Mappers by example: What are the colors of the squares?

input.csv (in HDFS)

color,	shape,	size
red,	circle,	3
red,	square,	5
blue,	oval,	1
green,	square,	3

0

red,circle,3

```
def map(key, value):  
    ...
```

zero or more output
key/value pairs

Mappers by example: What are the colors of the squares?

input.csv (in HDFS)

color,	shape,	size
red,	circle,	3
red,	square,	5
blue,	oval,	1
green,	square,	3

1

red, square, 5

```
def map(key, value):  
    ...
```

zero or more output
key/value pairs

Mappers by example: What are the colors of the squares?

input.csv (in HDFS)

color,	shape,	size
red,	circle,	3
red,	square,	5
blue,	oval,	1
green,	square,	3

1

red, square, 5

```
def map(key, value):  
    if value.shape == square:  
        emit(key, value.color)
```

key
1

value
red

Mappers by example: What are the colors of the squares?

input.csv (in HDFS)

color,	shape,	size
red,	circle,	3
red,	square,	5
blue,	oval,	1
green,	square,	3

2

blue,oval,1

```
def map(key, value):  
    if value.shape == square:  
        emit(key, value.color)
```

key
1

value
red

Mappers by example: What are the colors of the squares?

input.csv (in HDFS)

color,	shape,	size
red,	circle,	3
red,	square,	5
blue,	oval,	1
green,	square,	3

```
def map(key, value):  
    if value.shape == square:  
        emit(key, value.color)
```

green, square, 3

key

1
3

value

red
green

Mappers by example: What are the colors of the squares?

input.csv (in HDFS)

color,	shape,	size
red,	circle,	3
red,	square,	5
blue,	oval,	1
green,	square,	3

What if the data is huge?

```
def map(key, value):  
    if value.shape == square:  
        emit(key, value.color)
```

green, square, 3

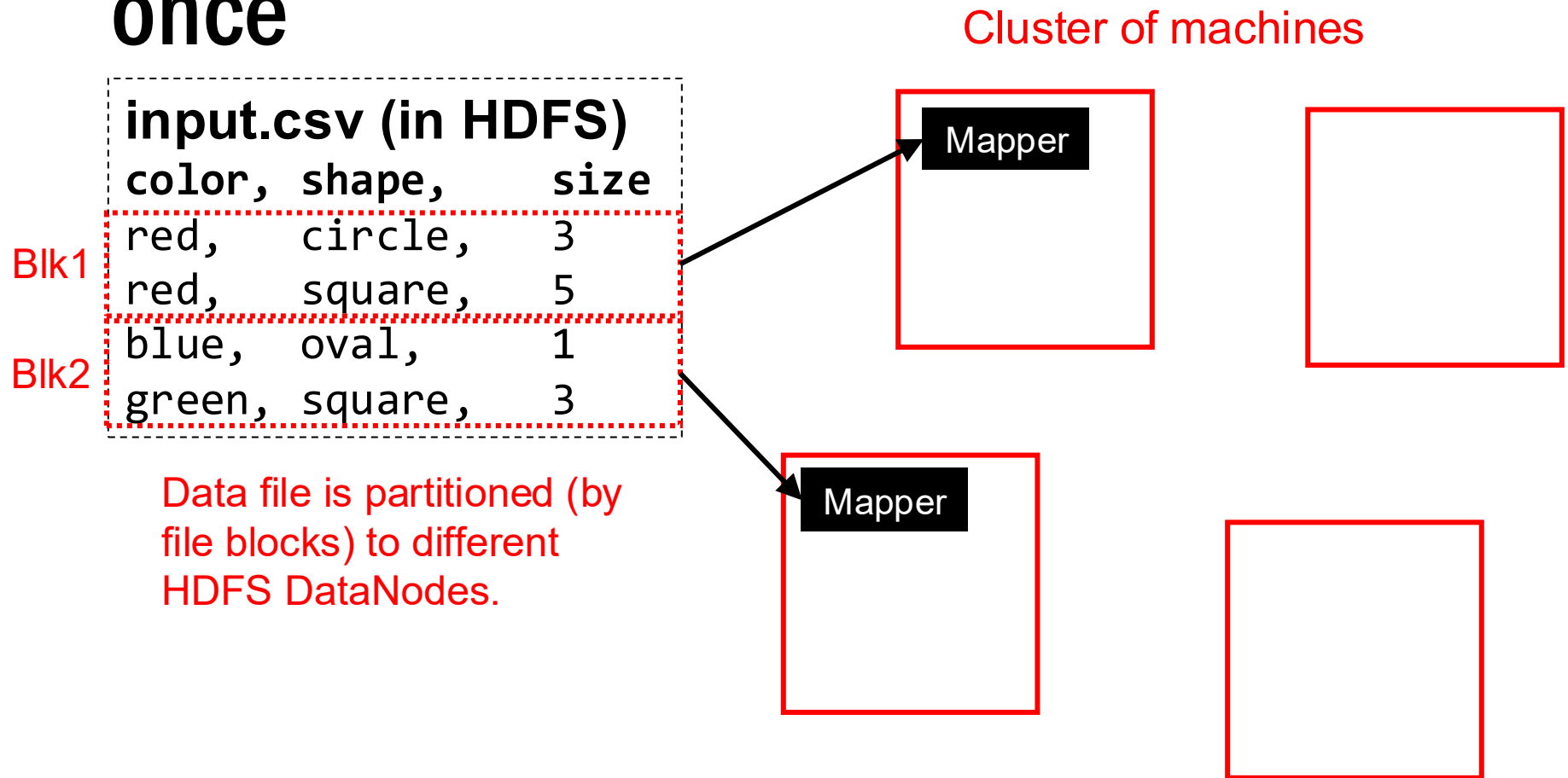
key

1
3

value

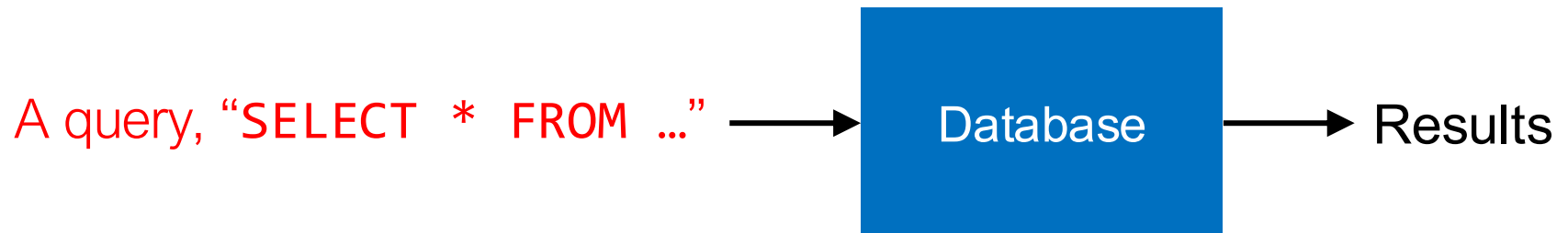
red
green

Mappers run on multiple machines at once

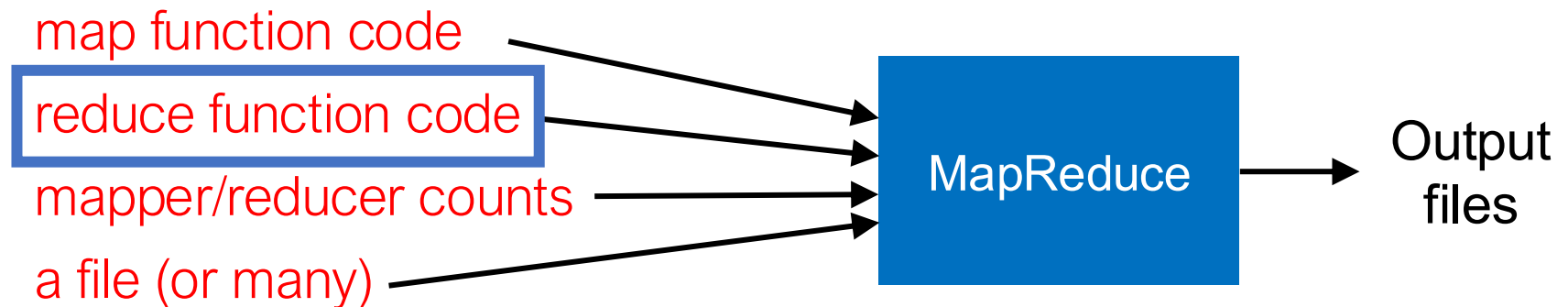


How does big data system answer questions?

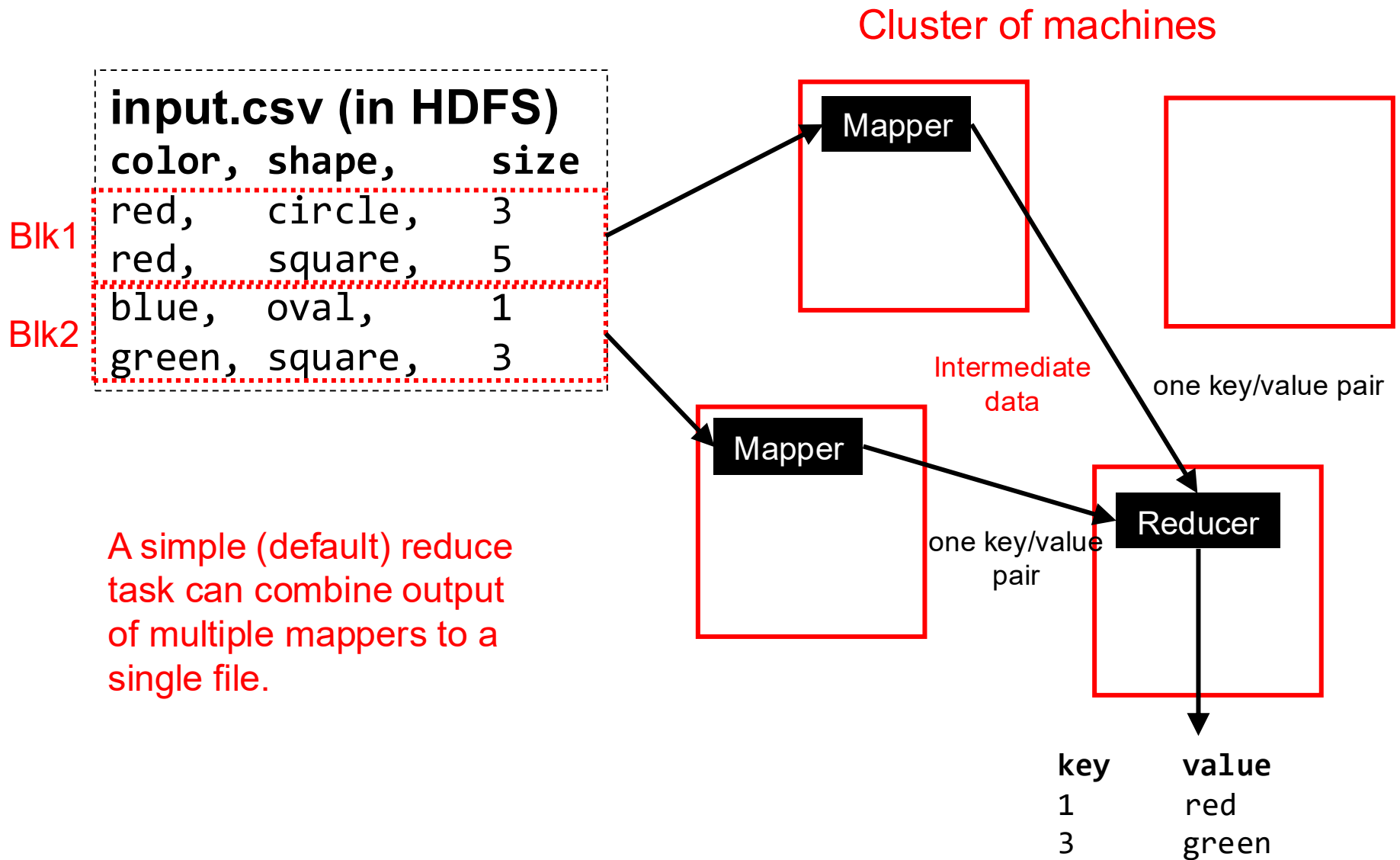
SQL



MapReduce



Reducers



Reducers

Reducers can output exactly their input,
OR have further computation.

```
def reduce(key, values):  
    for row in values:  
        emit(key, row)
```

Reducers

input.csv (in HDFS)

color	shape	size
red	circle	3
red	square	5
blue	oval	1
green	square	3

```
def map(key, value):  
    emit(value.color, value)
```

key	value
blue	blue, oval, 1
green	green, square, 3
red	red, circle, 3
red	red, square, 5

```
def reduce(key, values):  
    count = 0  
    for row in values:  
        count = count + 1  
    emit(key, count)
```

Intermediate data is **grouped**
and **sorted** by key.

Reduce will be called 3 times (once
for each group). The call could
happen in one reduce task (or be
split over many).

Reducers

input.csv (in HDFS)

color	shape	size
red	circle	3
red	square	5
blue	oval	1
green	square	3

```
def map(key, value):  
    emit(value.color, value)
```

key	value
blue	blue, oval, 1
green	green, square, 3
red	red, circle, 3
red	red, square, 5

```
def reduce(key, values):  
    count = 0  
    for row in values:  
        count = count + 1  
    emit(key, count)
```

Intermediate data is **grouped**
and **sorted** by key.

key	value
blue	1

Reducers

input.csv (in HDFS)

color	shape	size
red	circle	3
red	square	5
blue	oval	1
green	square	3

```
def map(key, value):  
    emit(value.color, value)
```

key	value
-----	-------

blue	blue, oval, 1
green	green, square, 3
red	red, circle, 3
red	red, square, 5

```
def reduce(key, values):  
    count = 0  
    for row in values:  
        count = count + 1  
    emit(key, count)
```

Intermediate data is **grouped**
and **sorted** by key.

key	value
blue	1
green	1

Reducers

input.csv (in HDFS)

color	shape	size
red	circle	3
red	square	5
blue	oval	1
green	square	3

```
def map(key, value):  
    emit(value.color, value)
```

key	value
-----	-------

blue	blue, oval, 1
green	green, square, 3
red	red, circle, 3
red	red, square, 5

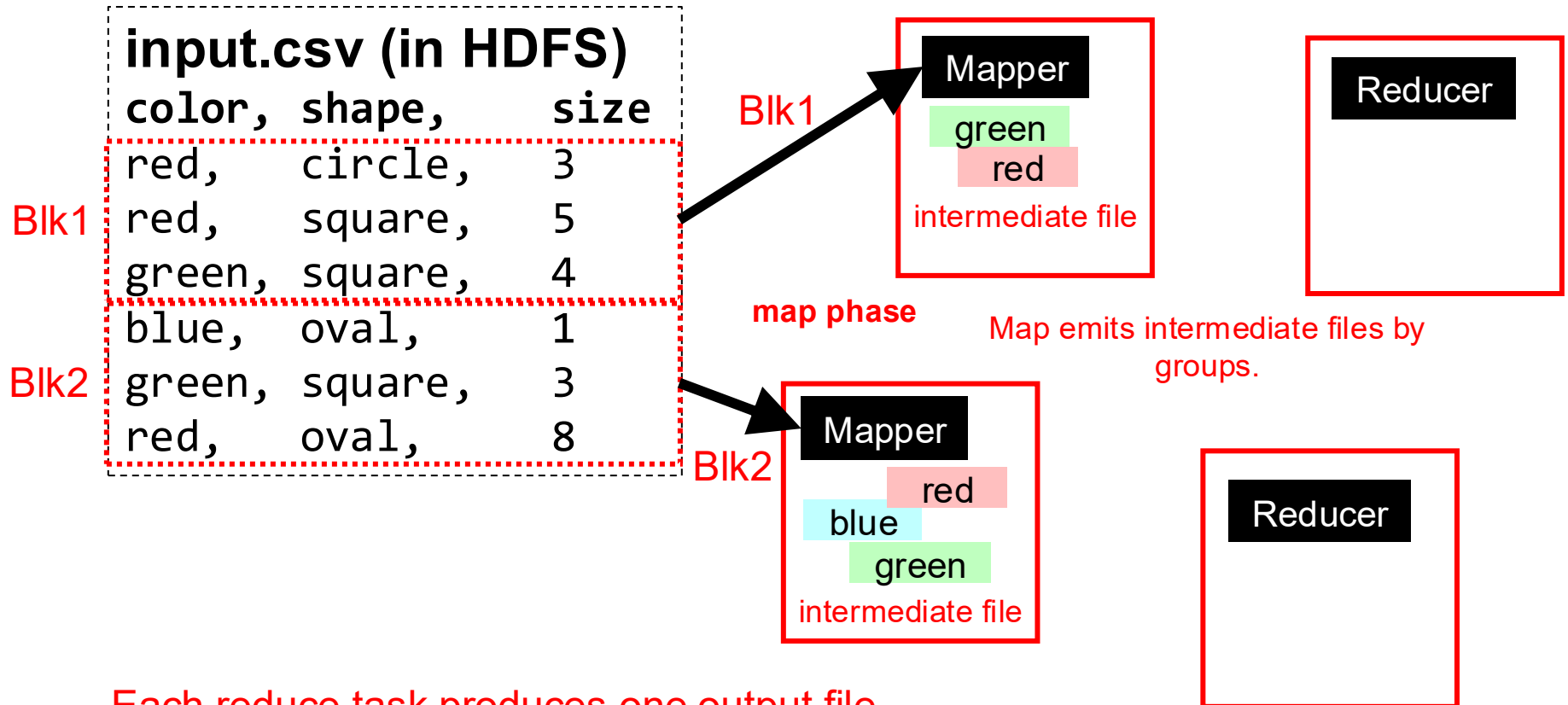
```
def reduce(key, values):  
    count = 0  
    for row in values:  
        count = count + 1  
    emit(key, count)
```

Intermediate data is **grouped**
and **sorted** by key.

key	value
blue	1
green	1
red	2

Multiple reducers (for big intermediate data)

Cluster of machines

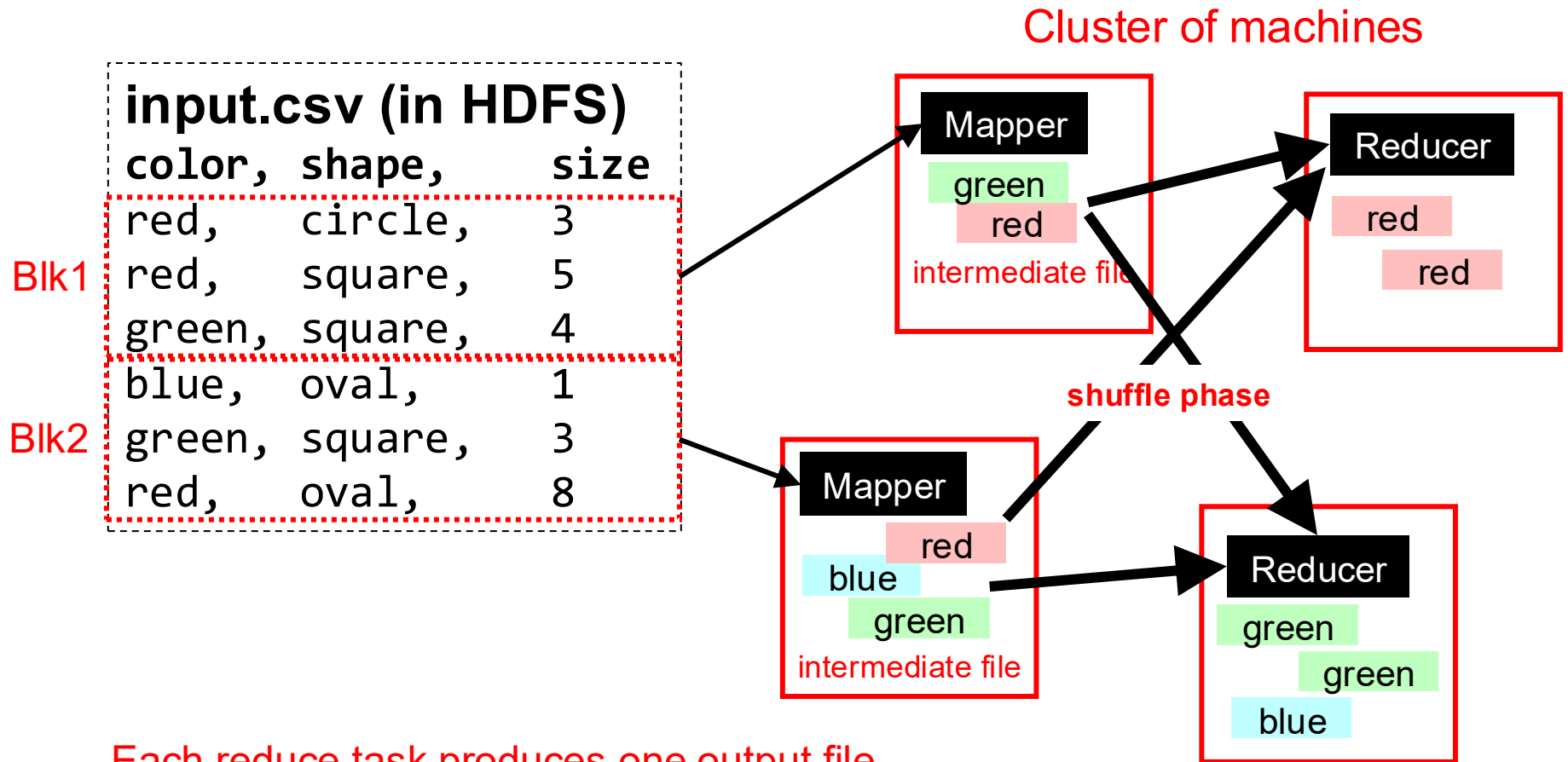


Each reduce task produces one output file.

A reduce task might take multiple keys.

All intermediate rows with the same key go to the same reducer.

Multiple reducers (for big intermediate data)



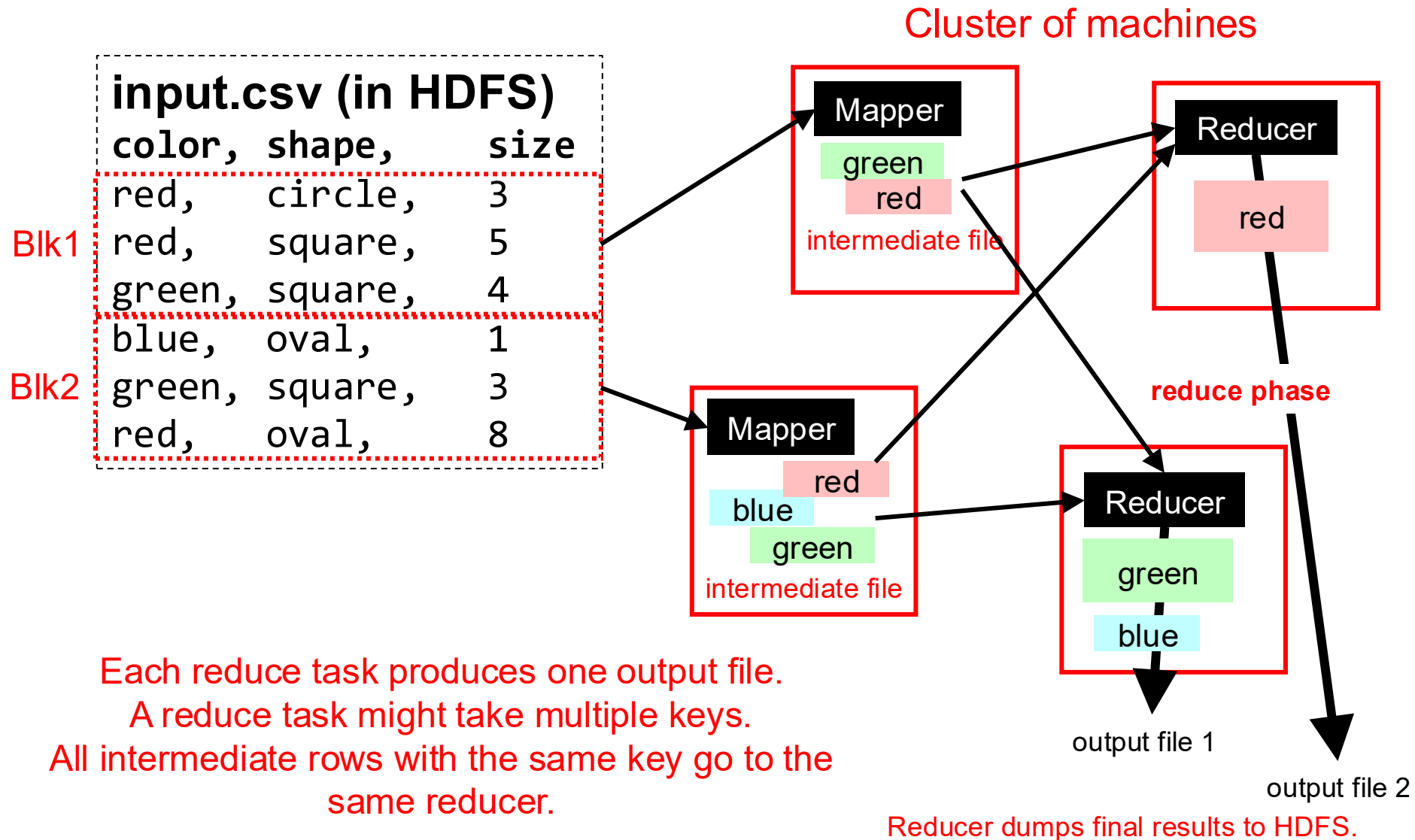
Each reduce task produces one output file.

A reduce task might take multiple keys.

All intermediate rows with the same key go to the same reducer.

Reducer collects all intermediate files of its assigned keys (groups).

Multiple reducers (for big intermediate data)

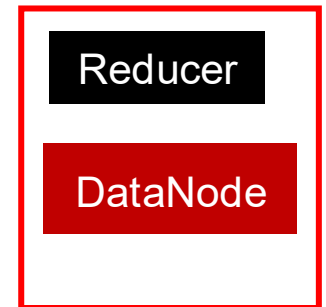
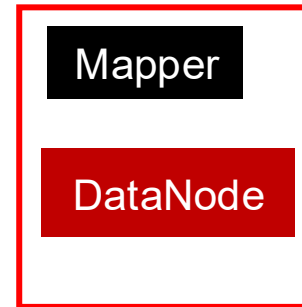


Data locality: Avoid network transfer

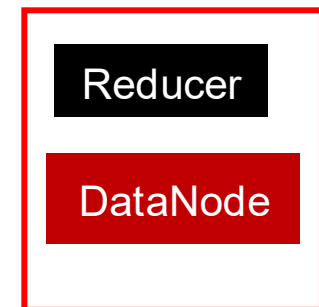
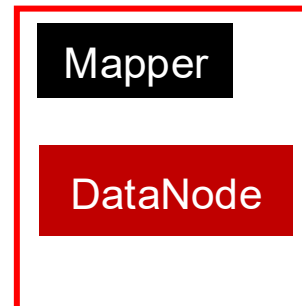
Cluster of machines

Run on same machines

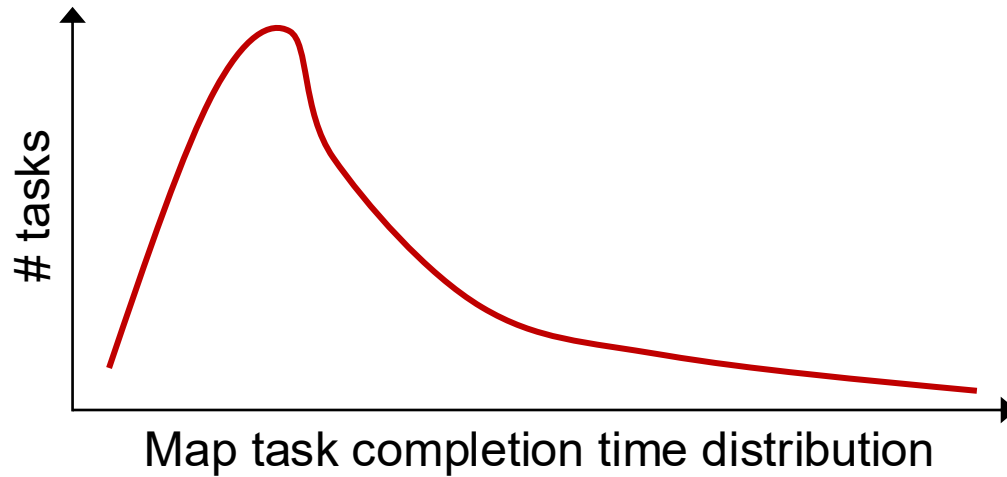
- Layered subsystems
- MapReduce executor
- HDFS DataNode



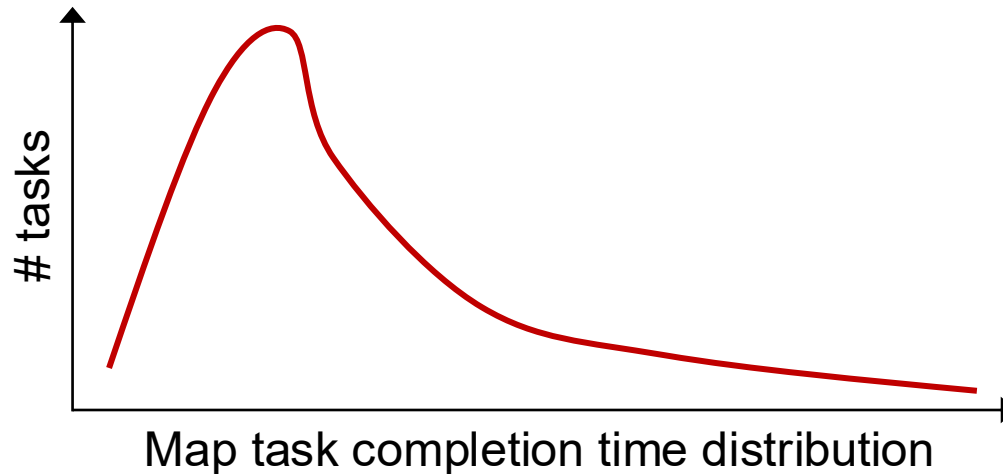
Try to run mappers on machine where DataNode has needed data. Uses local disk but not network.



Stragglers



Stragglers



- **Tail execution time** means some executors (always) finish late (**recall tail latency**)

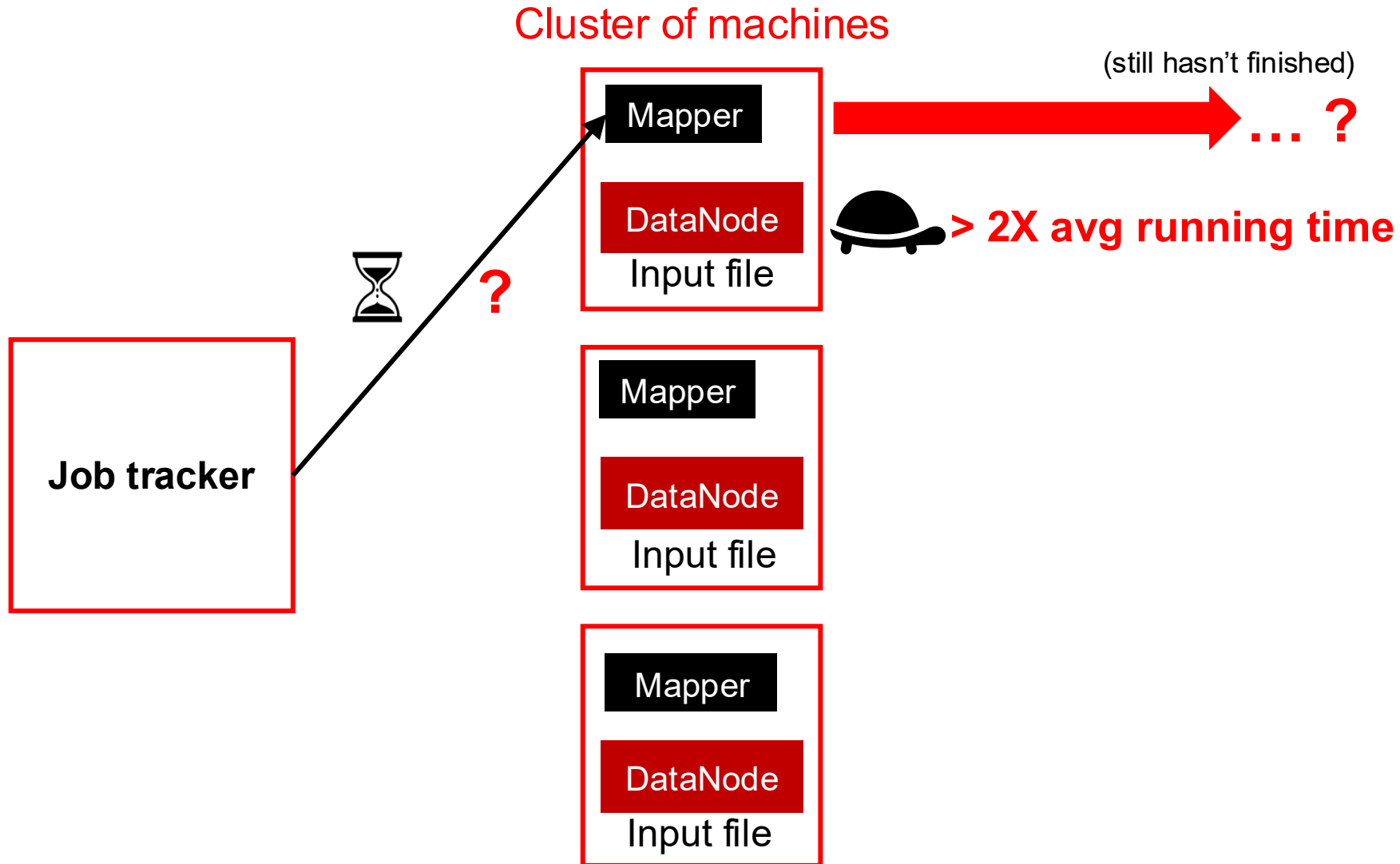
Q: How can MapReduce work around this?

- Hint: its approach to **fault-tolerance** provides the right tool

Resilience against stragglers?

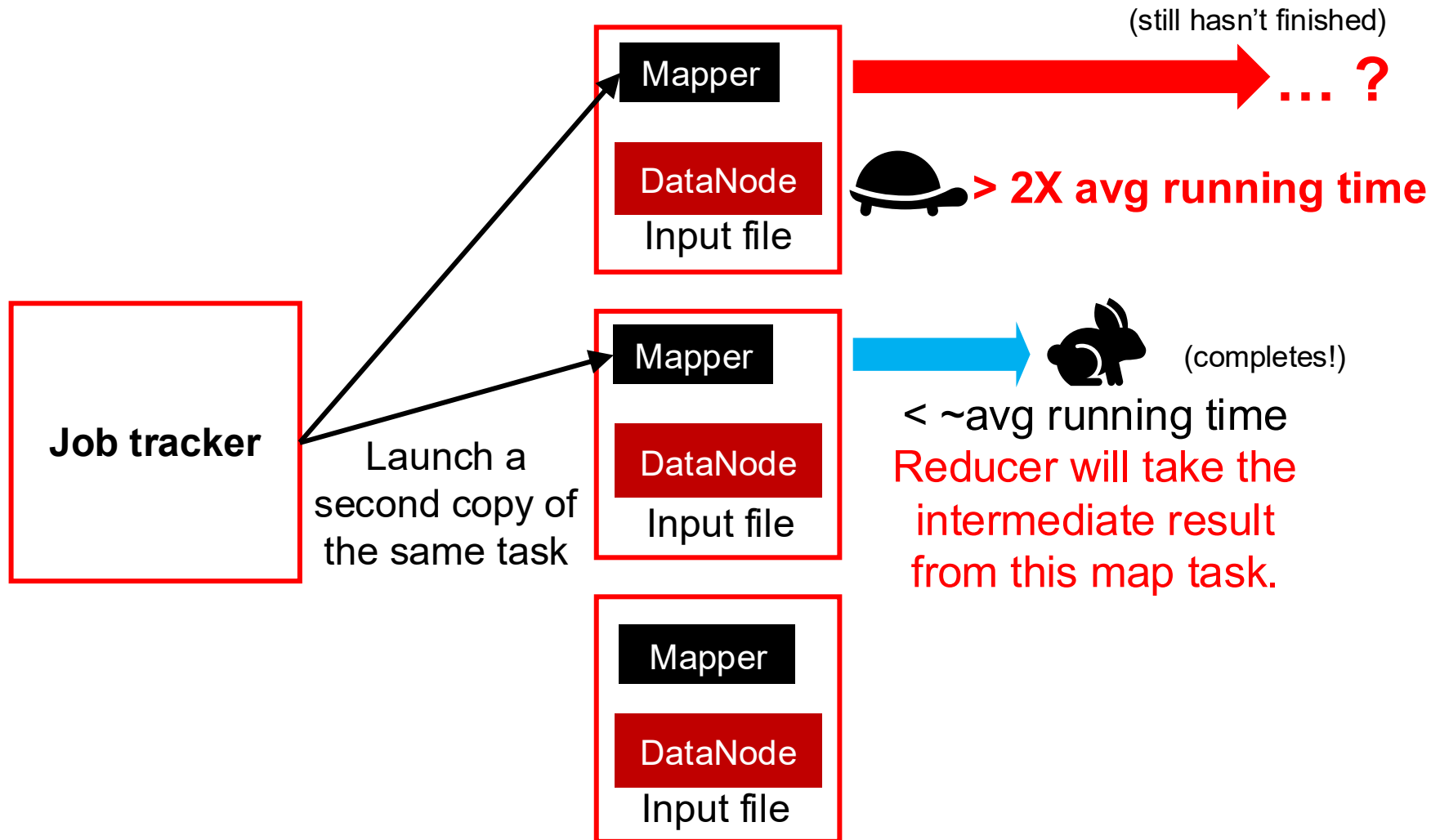
- If a task is going slowly (i.e., **straggler**):
 - Launch second copy of task (**backup task**) on another node
 - Take the output of whichever finishes first

Resilience against stragglers



Resilience against stragglers

Cluster of machines

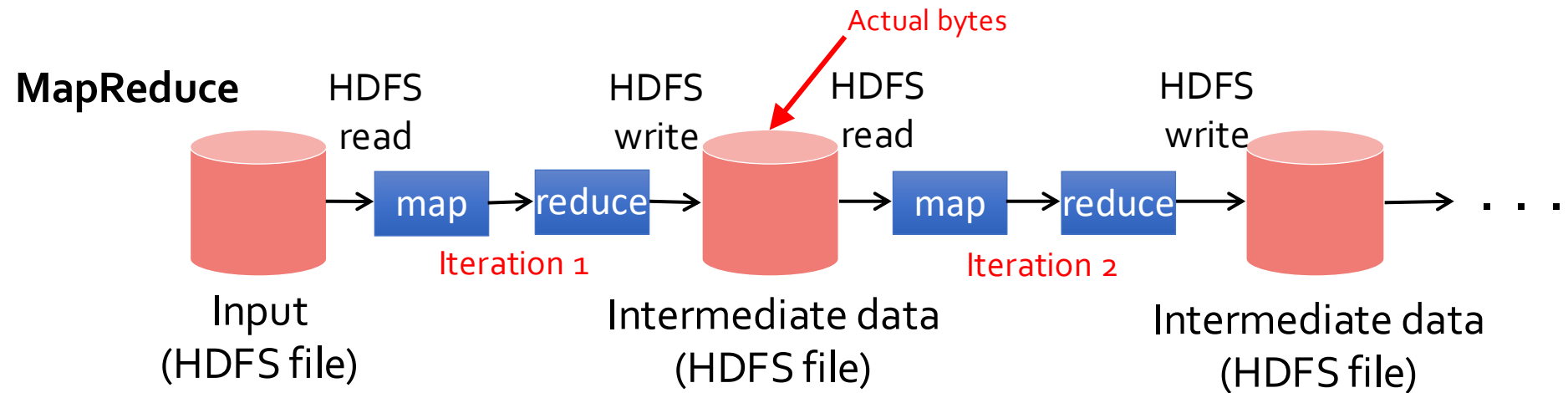


Would backup tasks cause correctness issue in MapReduce jobs?

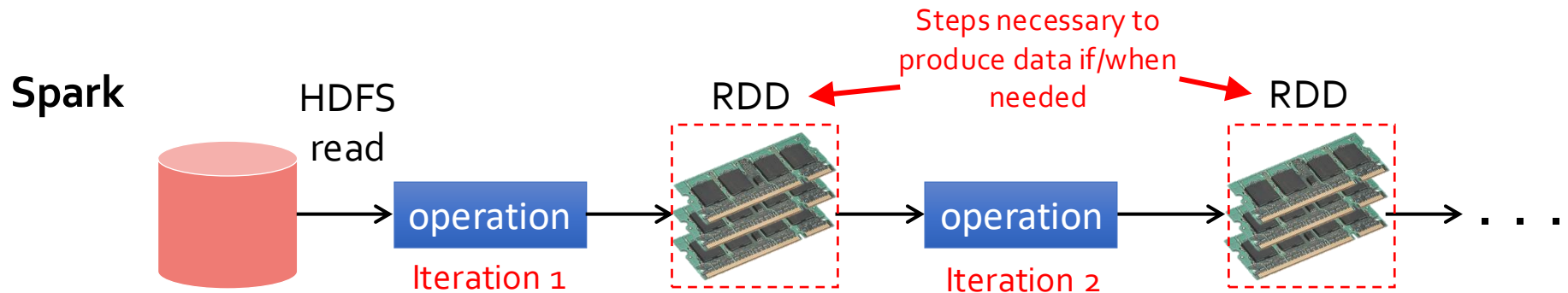
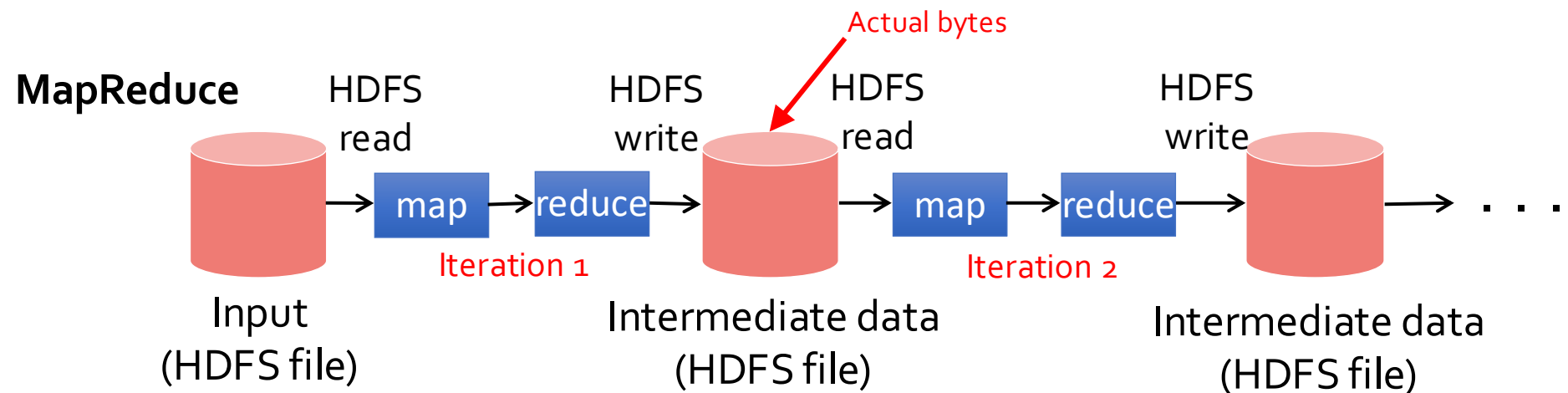
Outline

- GFS (HDFS)
- MapReduce
- Spark RDD

Intermediate data: MapReduce



Intermediate data: MapReduce vs. Spark



Resilient Distributed Datasets (RDD)

- **Data lineage:** Record series of operations on the data necessary to obtain results
- **Lazy evaluation:** Computation only done when results needed (to write file, make plot, etc.)
- **Immutability:** You can't change an RDD, but you can define a new one in terms of another

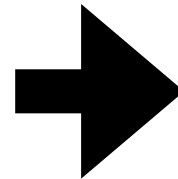
Data lineage: Transformations & Actions

```
data = [  
    ("A", 1),  
    ("B", 2),  
    ("A", 3),  
    ("B", 4)  
]
```

```
def mult2(row):  
    return (row[0], row[1]*2)  
  
def onlyA(row):  
    return row[0] == "A"
```

Goal: Get 2 times the second column wherever the first column is "A"

```
table = sc.parallelize(data)  
double = table.map(mult2)  
doubleA = double.filter(onlyA)  
doubleA.collect()
```



```
[('A', 2),  
 ('A', 6)]
```

The computation is a sequence of 4 operations. Operations come in two types:

- **Transformation:** Create a new RDD (lazy, so no execution yet). Here: `parallelize`, `map`, and `filter`.
- **Action:** Perform all operations in the graph to get an actual result. Here: `collect`.

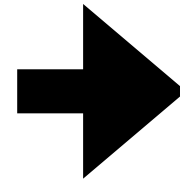
Data lineage: Transformations & Actions

```
data = [  
    ("A", 1),  
    ("B", 2),  
    ("A", 3),  
    ("B", 4)  
]
```

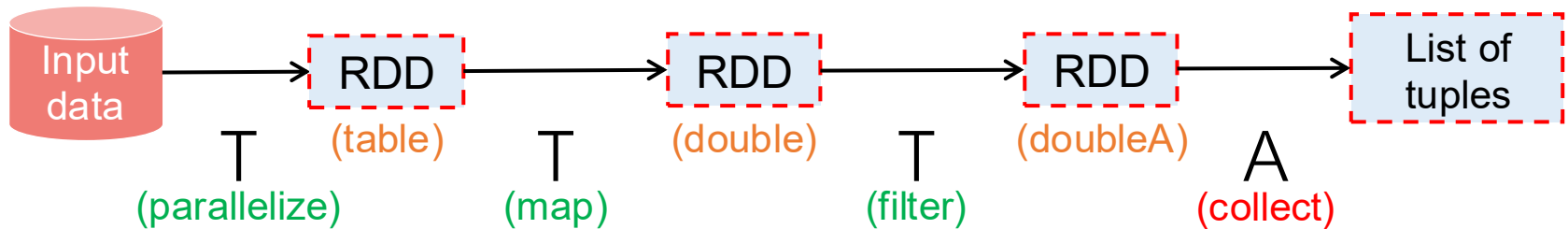
```
def mult2(row):  
    return (row[0], row[1]*2)  
  
def onlyA(row):  
    return row[0] == "A"
```

Goal: Get 2 times the second column wherever the first column is "A"

```
table = sc.parallelize(data)  
double = table.map(mult2)  
doubleA = double.filter(onlyA)  
doubleA.collect()
```



```
[('A', 2),  
 ('A', 6)]
```



Q: Are there alternative paths you could create from the start to end node?

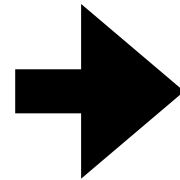
Optimization

Transformation vs. action:

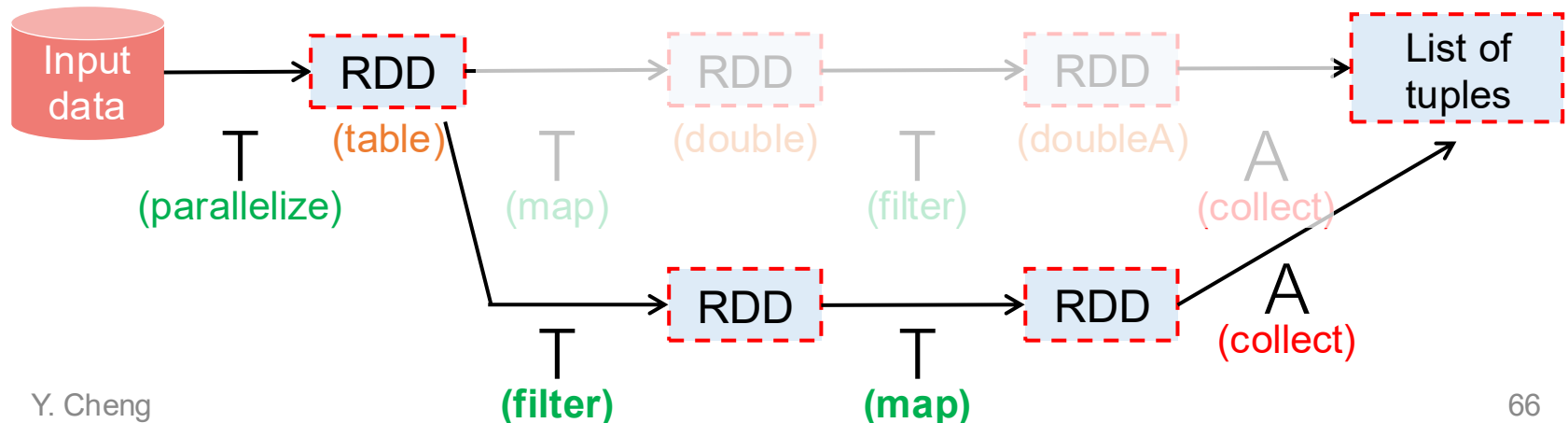
- **Transformation:** intermediate results (means to an end)
- **Action:** Final results we care about
- This distinction creates opportunities for **optimization** (choosing a **more efficient sequence of transformations** to get the same result + **pipelining the compute**)

Goal: Get 2 times the second column wherever the first column is "A"

```
table = sc.parallelize(data)
double = table.map(mult2)
doubleA = double.filter(onlyA)
doubleA.collect()
```

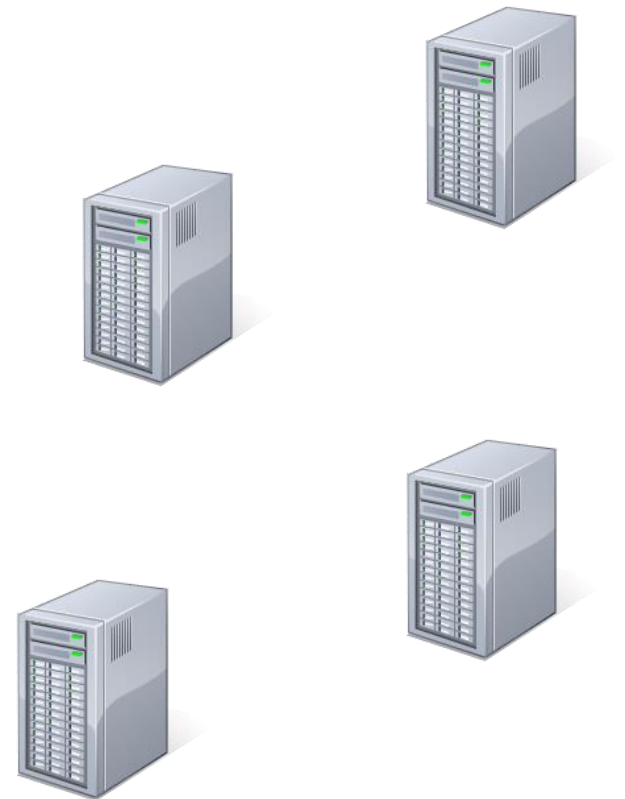


```
[('A', 2),  
 ('A', 6)]
```



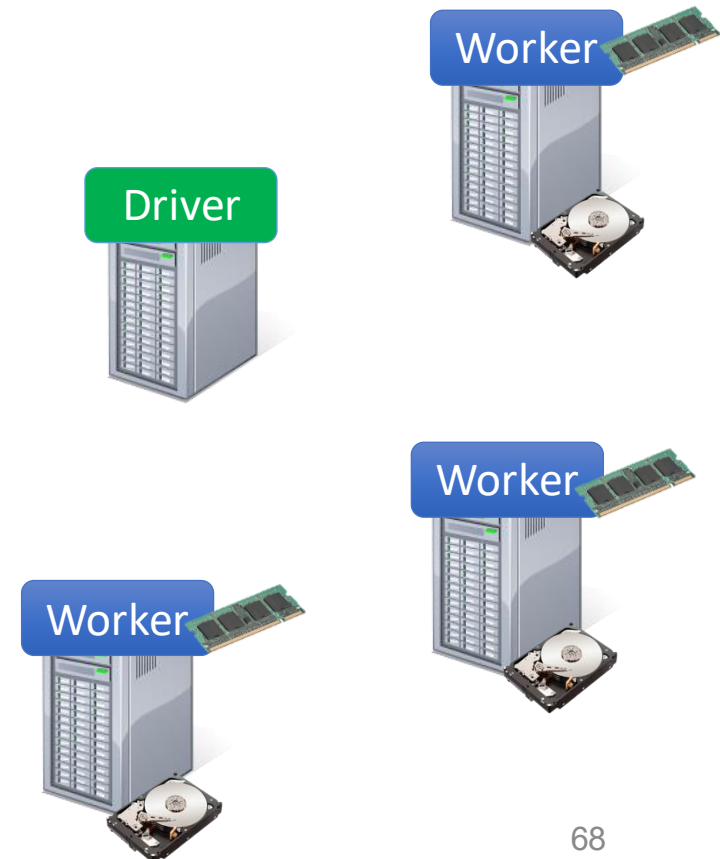
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns



Interactive debugging (control & data flow)

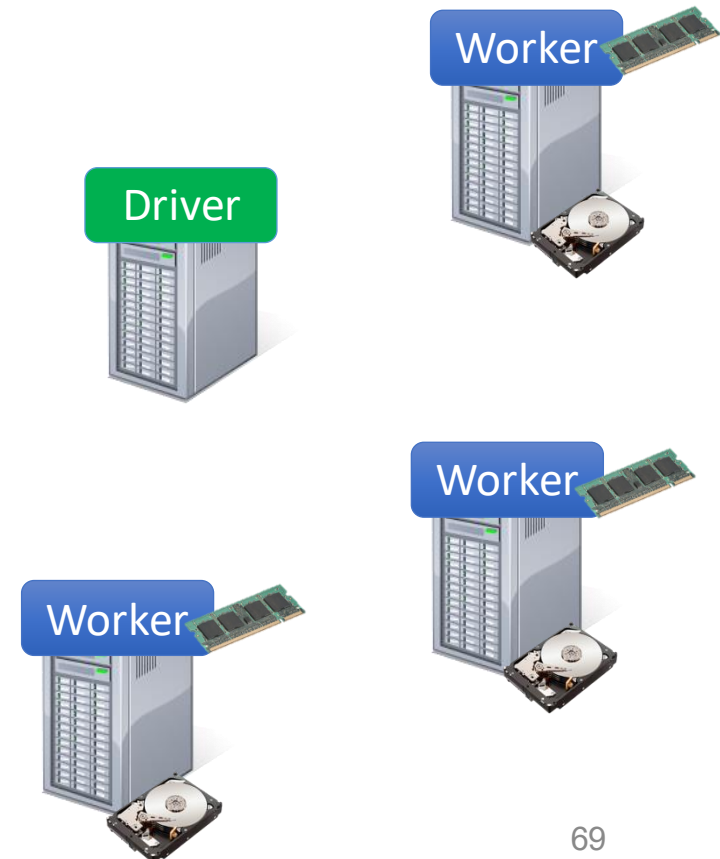
Load input data from an HDFS file into memory, then interactively search for various patterns



Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
```



Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
```

Base RDD

Driver

Worker

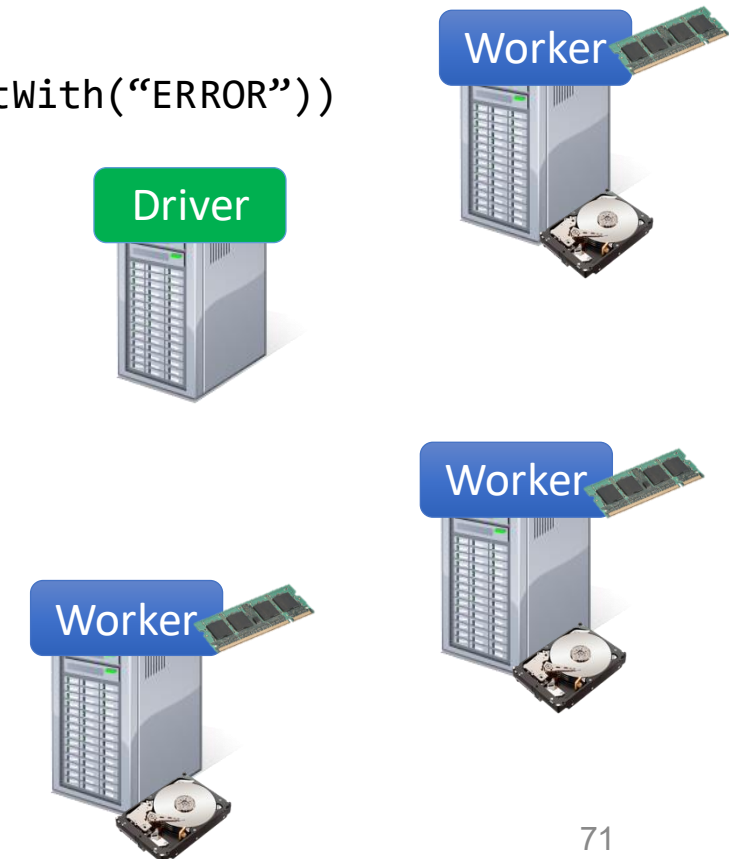
Worker

Worker

Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")  
errors = lines.filter(lambda line: line.startswith("ERROR"))
```



Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")  
errors = lines.filter(lambda line: line.startsWith("ERROR"))
```

Transformed RDD

Driver

Worker

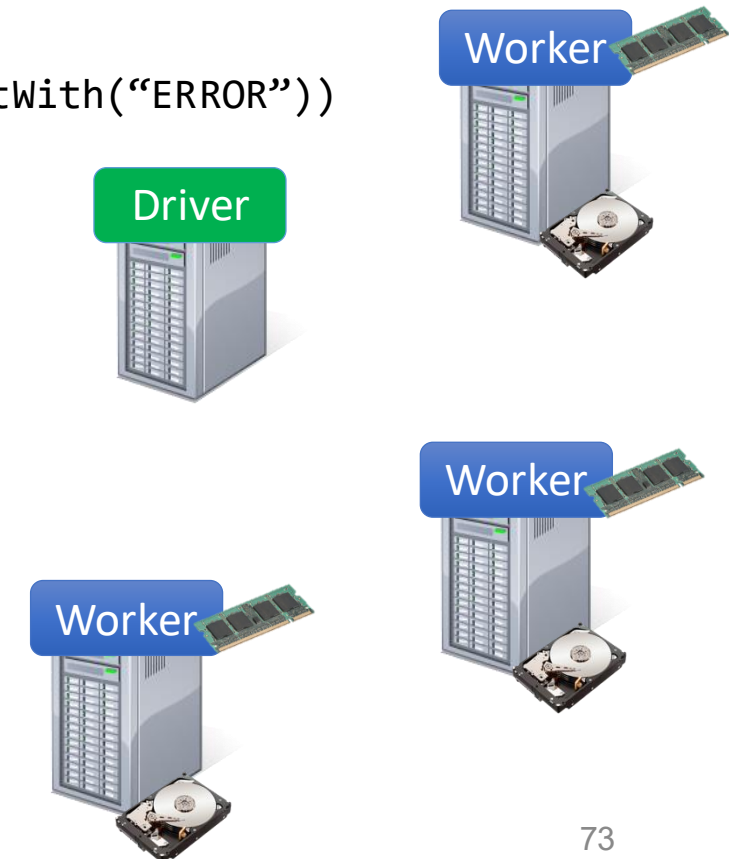
Worker

Worker

Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")  
errors = lines.filter(lambda line: line.startswith("ERROR"))
```



Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")  
errors = lines.filter(lambda line: line.startswith("ERROR"))  
messages = errors.map(lambda error: error.split('\t')[2])
```

Driver

Worker

Worker

Worker

Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
```

Another Transformed RDD

Driver

Worker

Worker

Worker

Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

Driver

Worker

Worker

Worker

Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

Tell workers to cache it to memory for reuse

Driver

Worker

Worker

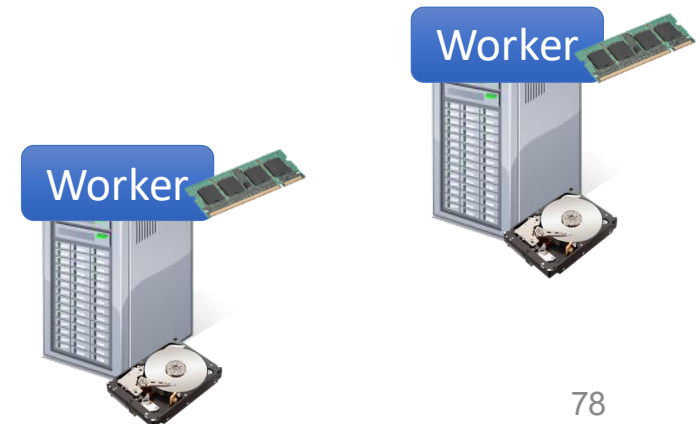
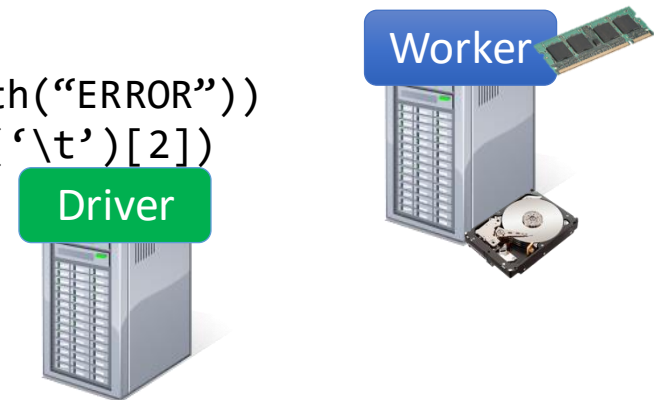
Worker

Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```



Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

Action

Driver

Worker

Worker

Worker

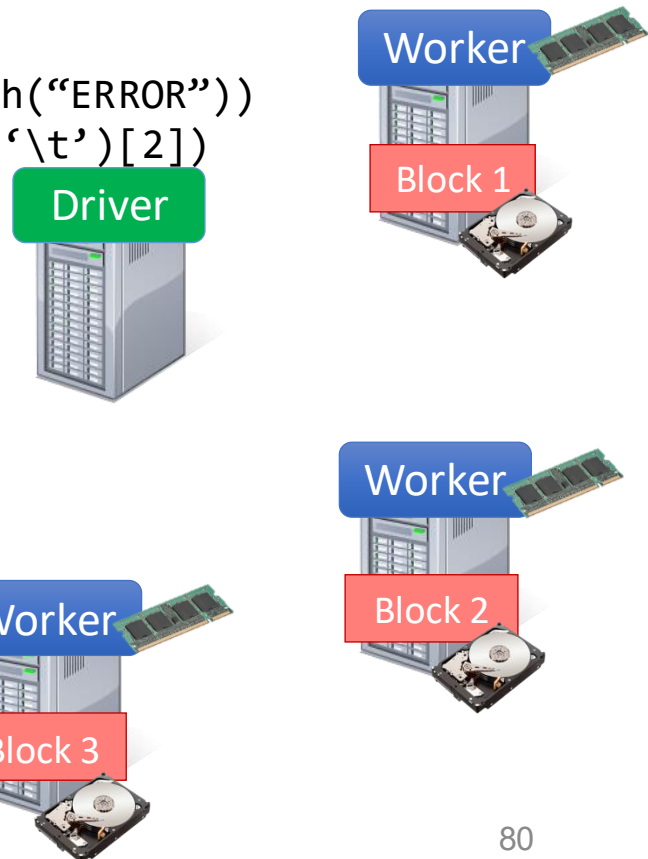
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

Action



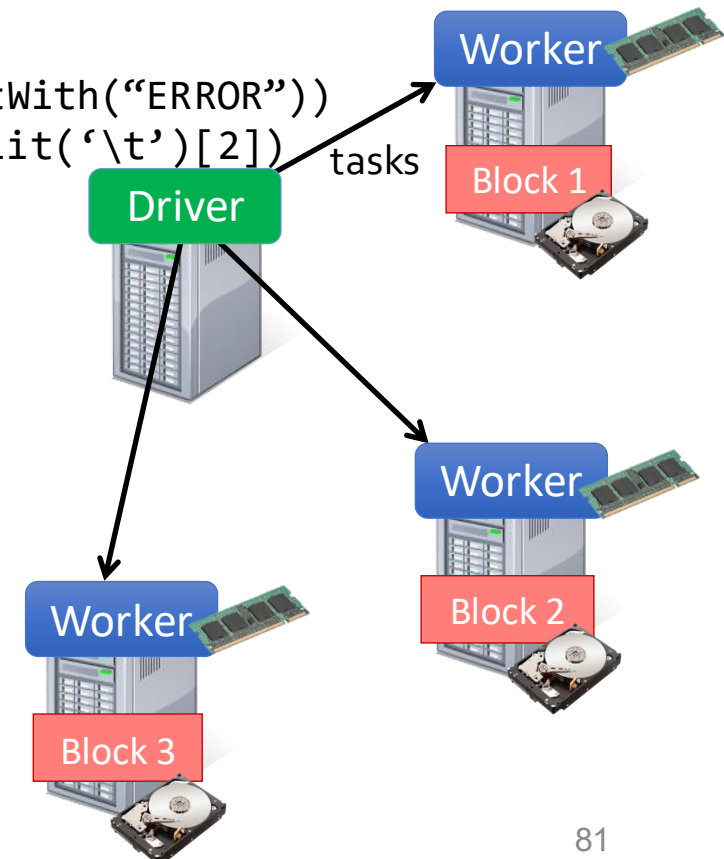
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startsWith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

Action



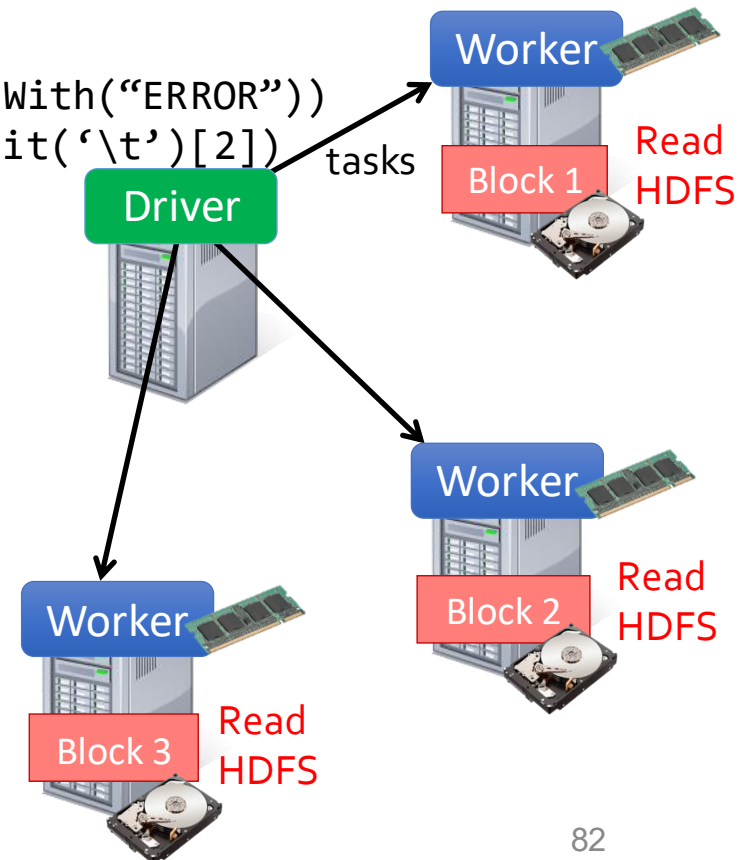
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startsWith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

Action



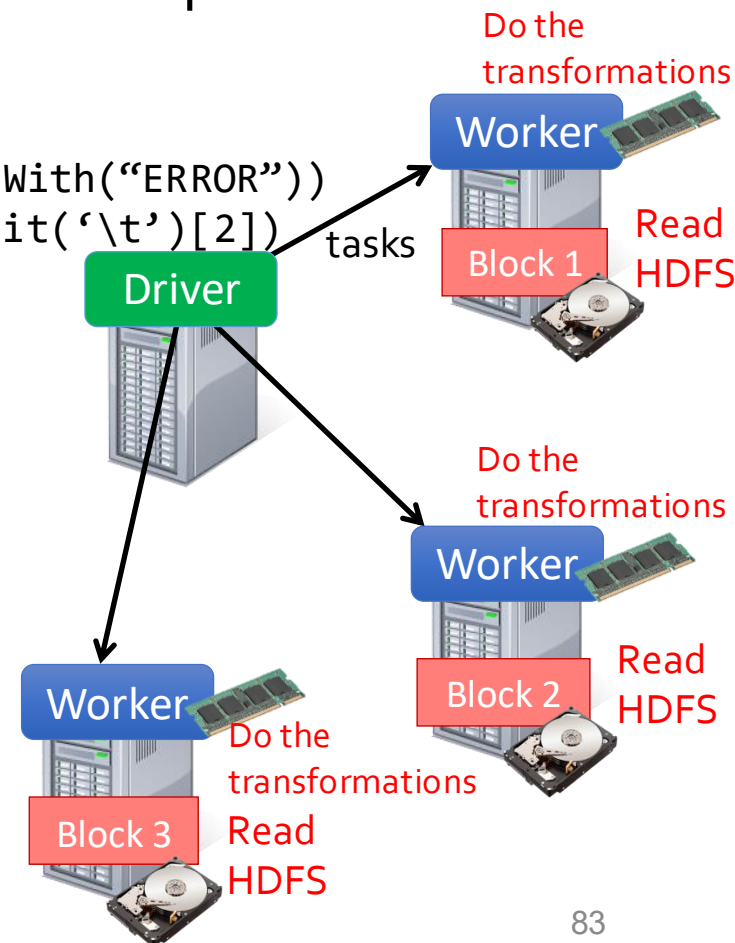
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startsWith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

Action



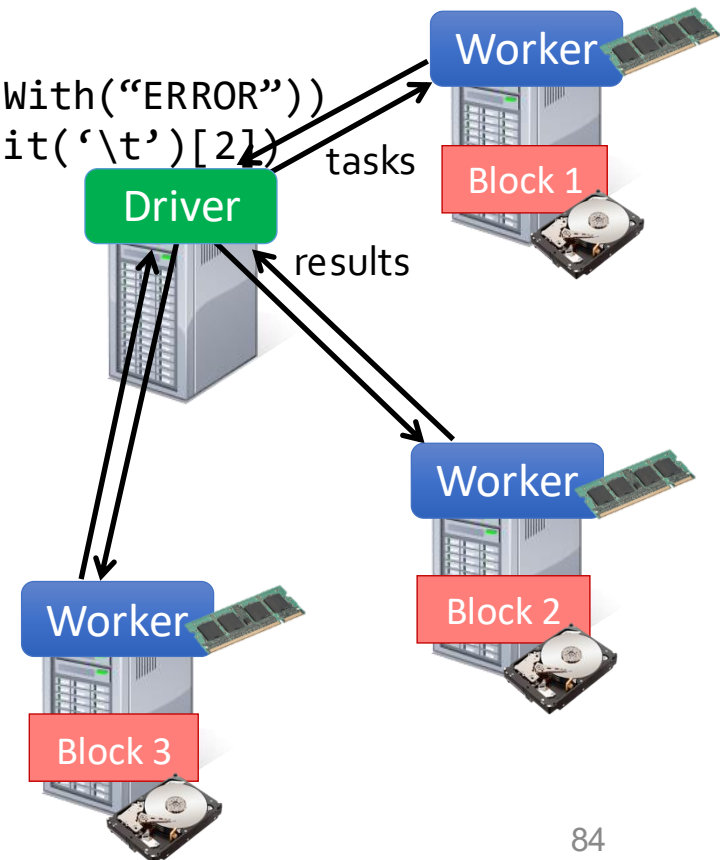
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startsWith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
               .count()
```

Action



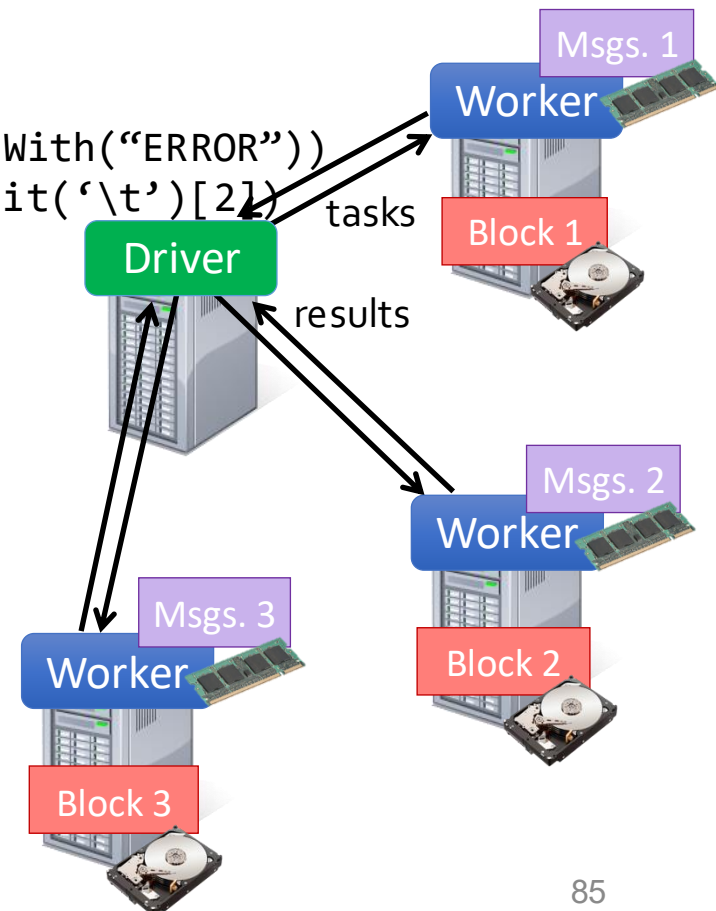
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
               .count()
```

Action

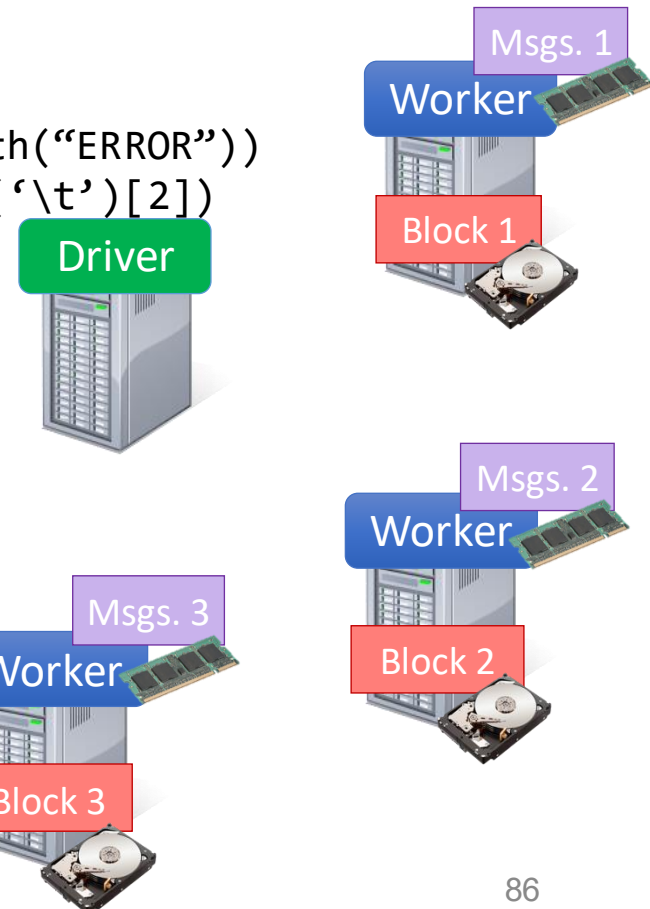


Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
            .count()
```



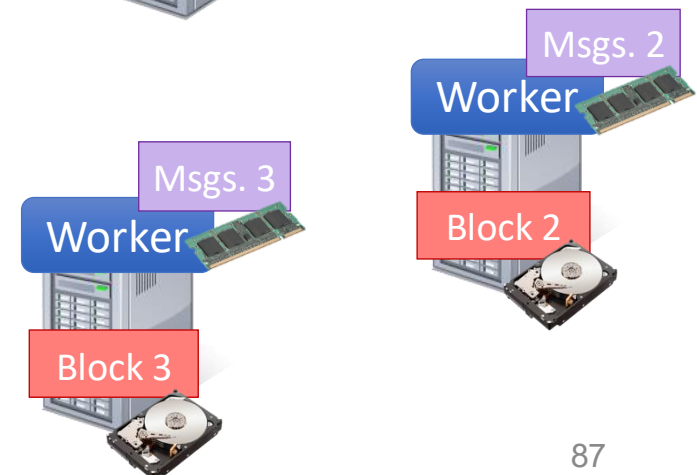
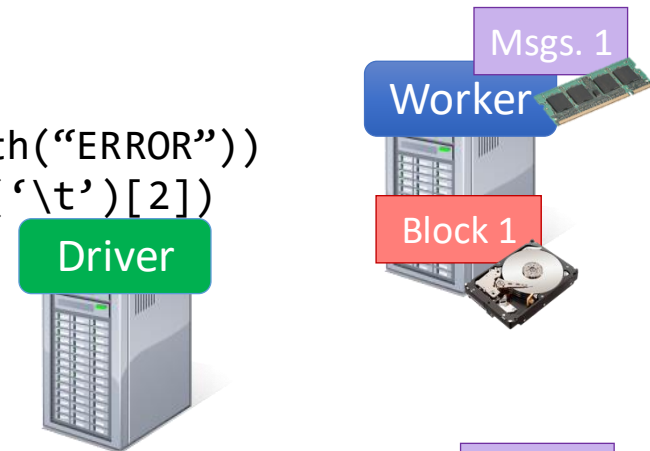
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

```
messages.filter(lambda line: "HDFS" in line)
           .count()
```



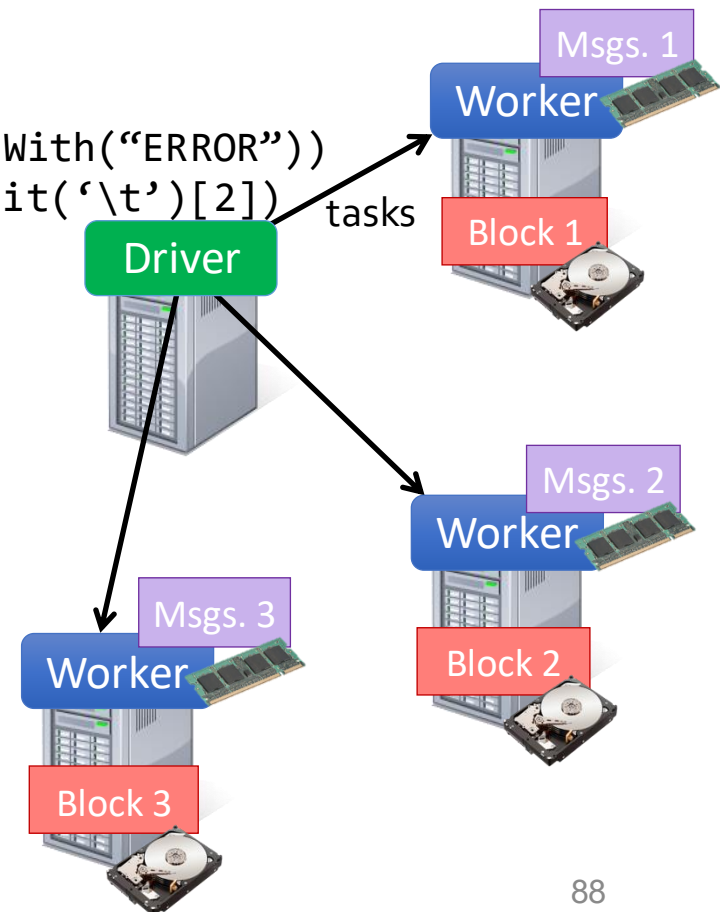
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

```
messages.filter(lambda line: "HDFS" in line)
           .count()
```



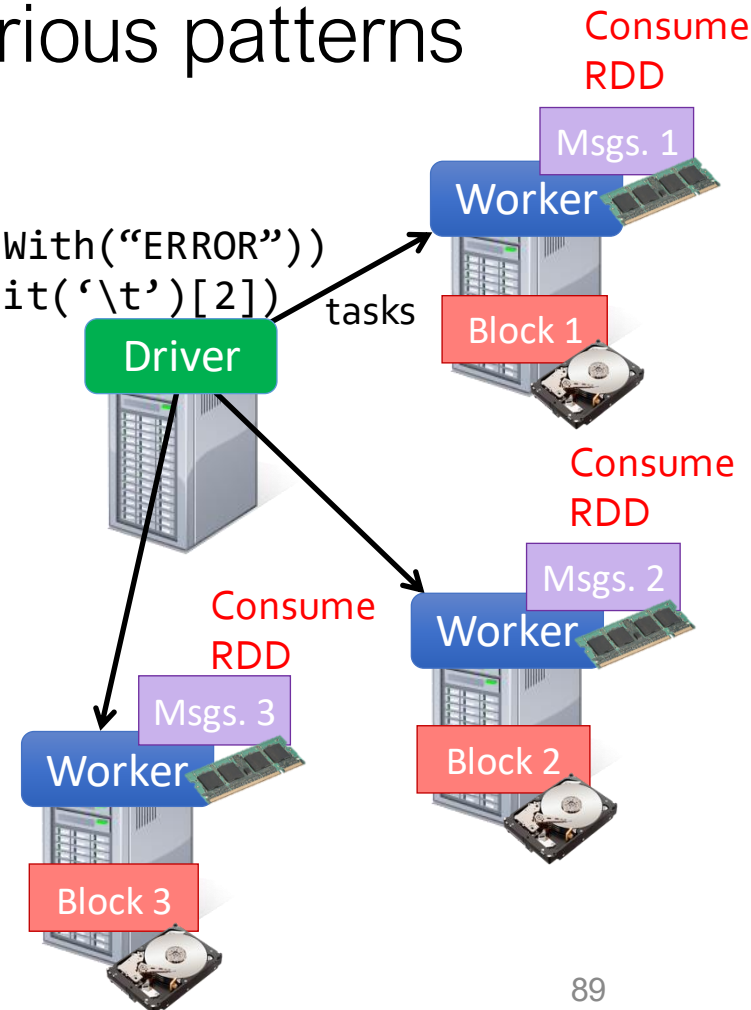
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

```
messages.filter(lambda line: "HDFS" in line)
           .count()
```



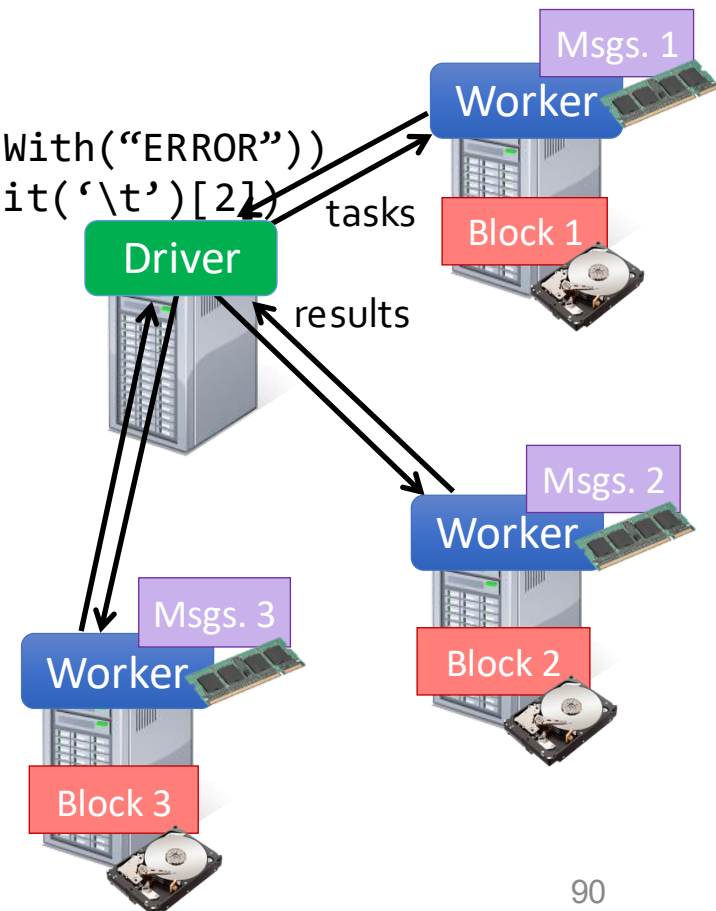
Interactive debugging (control & data flow)

Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

```
messages.filter(lambda line: "HDFS" in line)
           .count()
```



Interactive debugging (control & data flow)

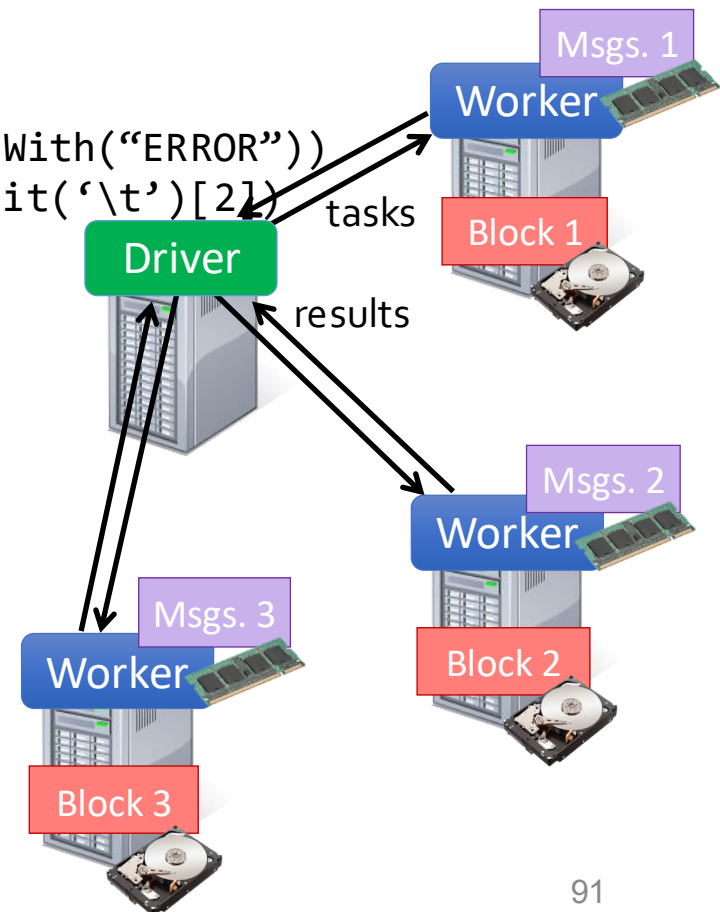
Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

```
messages.filter(lambda line: "HDFS" in line)
           .count()
```

Result: full-text search of Wikipedia in
<1 sec (vs. 20 sec for on-disk data)



Interactive debugging (control & data flow)

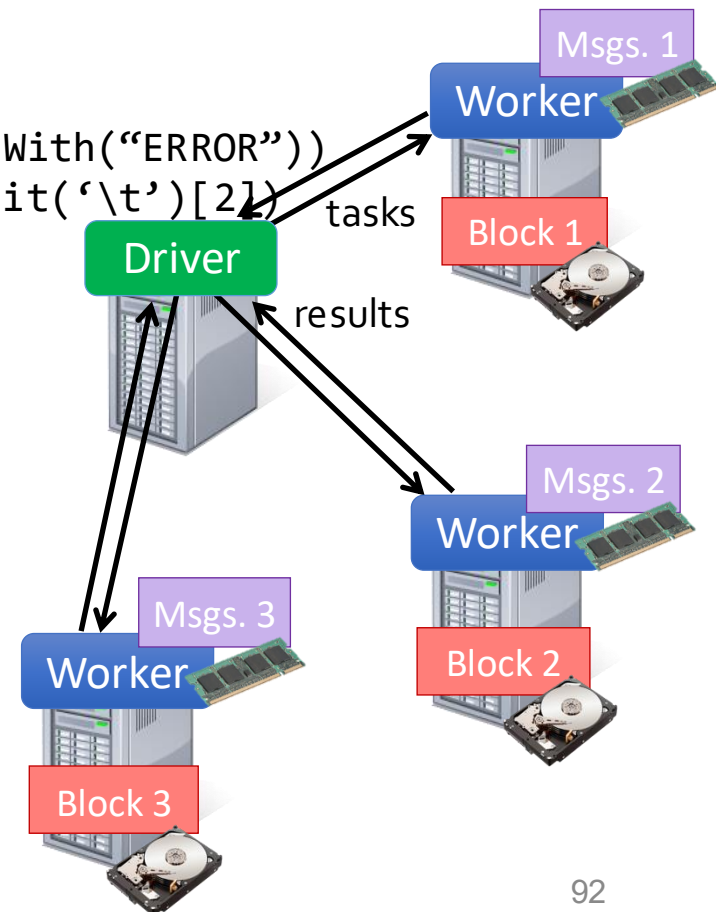
Load input data from an HDFS file into memory, then interactively search for various patterns

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda line: line.startswith("ERROR"))
messages = errors.map(lambda error: error.split('\t')[2])
messages.cache()
```

```
messages.filter(lambda line: "MySQL" in line)
           .count()
```

```
messages.filter(lambda line: "HDFS" in line)
           .count()
```

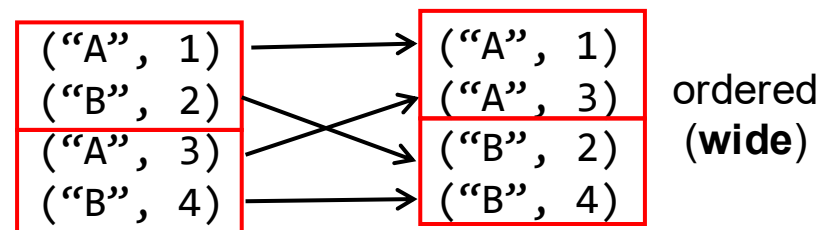
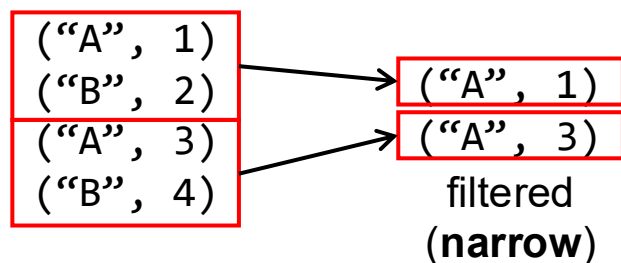
Result: scaled to 1 TB data in 5-7 sec
(vs. 170 sec for on-disk data)



Transformations: Narrow vs. Wide

- Any transformation where a single output partition can be computed from a single input partition is a **narrow transformation**.
- Others are **wide transformations**.

```
data = [("A", 1), ("B", 2), ("A", 3), ("B", 4),]  
table = sc.parallelize(data, 2)  
filtered = table.filter(lambda row: row[0] == "A")  
ordered = table.sortBy(lambda row: row[0])
```



- Wide transformations often require **network resources**. Unless all input partitions are on the same machine, some will need to be transferred.

Join and partitioning

Join and partitioning (best case)

Computer 1	Alice	5	⋈	Alice	F	=	Alice	5	F
	Bob	6		Bob	M		Bob	6	M
Computer 2	Claire	4		Claire	F		Claire	4	F

Join and partitioning (worst case)

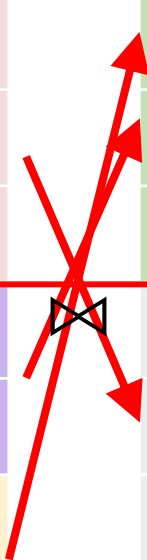
Computer 1	Alice	5	⋈	Alice	F	=	Alice	5	F
	Bob	6		Bob	M		Bob	6	M
Computer 2	Claire	4		Claire	F		Claire	4	F

Computer 1	A	5	⊗	C	5
	A	2		B	2
	A	3		A	3
Computer 2	B	4	B	4	
	B	1	A	1	
	C	6	B	6	
	C	8	C	8	

If partitioning doesn't match, then need to shuffle (**all-to-all network communication**) to match pairs.

Join and partitioning (worst case)

Computer 1	Alice	5	⋈	Alice	F	=	Alice	5	F
	Bob	6		Bob	M		Bob	6	M
Computer 2	Claire	4		Claire	F		Claire	4	F

Computer 1	A	5		C	5
	A	2		B	2
	A	3		A	3
Computer 2	B	4		B	4
	B	1		A	1
	C	6		B	6
	C	8		C	8

If partitioning doesn't match, then need to shuffle (**all-to-all network communication**) to match pairs.

Join and partitioning (optimization)

Computer 1	Alice	5	$\bowtie$	Alice	F	=	Alice	5	F
	Bob	6		Bob	M		Bob	6	M
Computer 2	Claire	4		Claire	F		Claire	4	F

Observation: What if the two tables are partitioned the same way?

Computer 1	A	5	$\bowtie$	A	3
	A	2		A	1
	A	3			
Computer 2	B	4		B	2
	B	1		B	4
	C	6		B	6
	C	8		C	5
				C	8

`partitionBy()` is specific to key-value pair RDDs. It is used to partition RDDs based on keys, by default using a **hash partitioner**.

Example: PageRank

Example: PageRank

1. Start each page with a rank of 1
2. On each iteration, update each dest page's rank to $\sum_{i \in \text{neighbors}} \text{rank}_{\text{neighbor}_i} / |\text{neighbors}_i|$

links = // RDD of (url, neighbors) pairs

ranks = // RDD of (url, rank) pairs

```
for (i <- 1 to ITERATIONS) {  
  ranks = links.join(ranks).flatMap {  
    (url, (links, rank)) =>  
      links.map(dest => (dest, rank/links.size))  
  }.reduceByKey(_ + _)  
}
```

Example: PageRank

1. Start each page with a rank of 1
2. On each iteration, update each dest page's rank to $\sum_{i \in \text{neighbors}} \text{rank}_{\text{neighbor}_i} / |\text{neighbors}_i|$

`RDD[(URL, Seq[URL])]`

`links = // RDD of (url, neighbors) pairs`

`ranks = // RDD of (url, rank) pairs` ← `RDD[(URL, Rank)]`

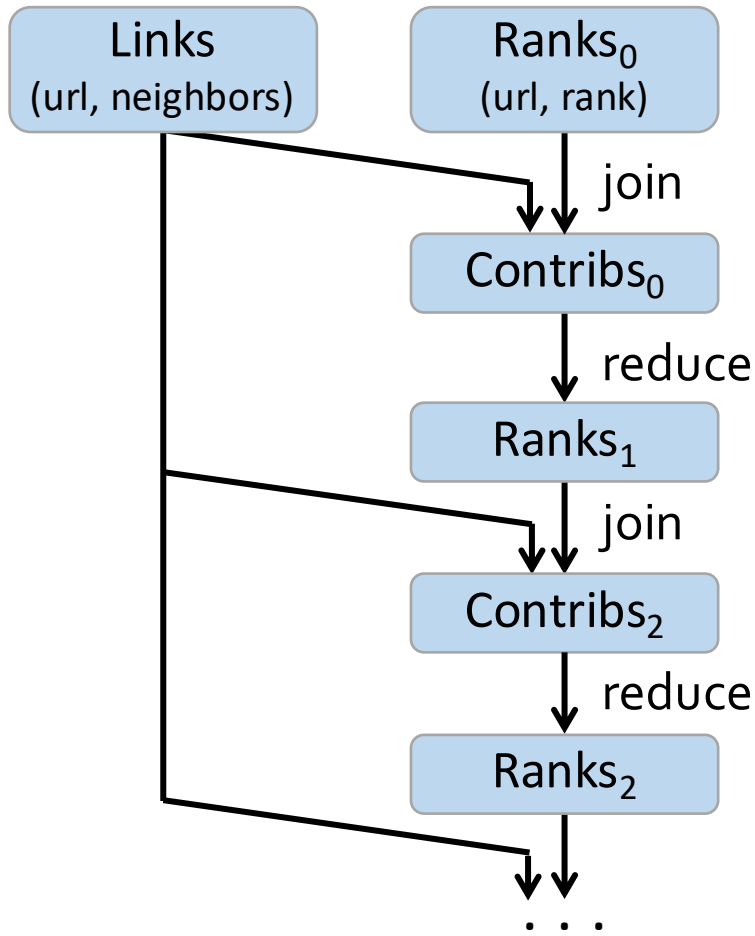
```
for (i <- 1 to ITERATIONS) {  
  ranks = links.join(ranks).flatMap {  
    (url, (links, rank)) =>  
      links.map(dest => (dest, rank/links.size))  
  }.reduceByKey(_ + _)  
}
```

← `RDD[(URL, (Seq[URL], Rank))]`

For each neighbor in links emits (URL, RankContrib)

Reduce to `RDD[(URL, Rank)]`

Optimizing placement

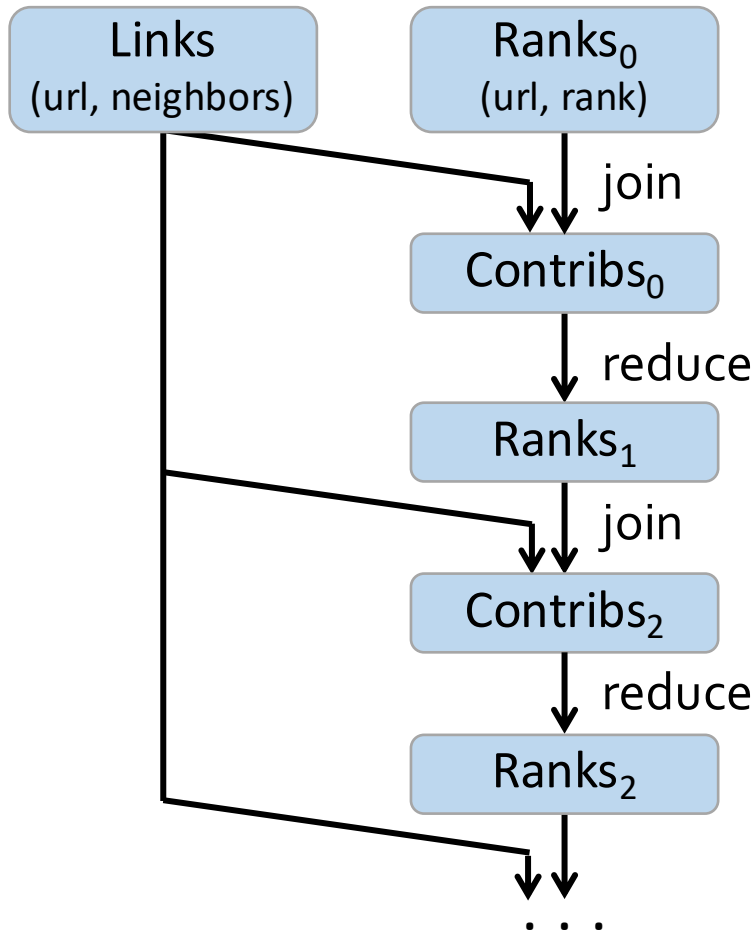


- `links` & `ranks` repeated joined
- Can co-partition them (e.g., hash both on source URLs) to avoid shuffles

`links = links.partitionBy(N)`

`ranks = ranks.partitionBy(N)`

Optimizing placement



- **links** & **ranks** repeated joined
- Can co-partition them (e.g., hash both on source URLs) to avoid shuffles

`links = links.partitionBy(N)`

`ranks = ranks.partitionBy(N)`

Q1: Should we apply `.persist()` to **links** or **ranks**?

Q2: Where might we have placed `.persist()`?